基于 SR-LUT 的双 FPGA 图像超分辨率系统

摘要

随着科技设备的不断更新，用户对于日常图像设备的清晰度和实时性要求越来越高。当前市面上常用的传统插值算法虽然简单易行，但精度较低，图像放大时容易出现边缘锯齿和轮廓模糊。超分辨率重建能够改善显示效果，但复杂神经网络对计算、存储与内存带宽的需求，又限制了其在嵌入式终端上的部署。因此，需要在有限硬件资源下兼顾图像重建质量与交互显示。

针对上述问题，本项目设计并实现了**基于 SR-LUT 的双 FPGA 图像超分辨率系统**，将学习得到的局部映射存入查找表，以整数寻址和插值完成在线推理，构建以下两级处理架构：

1. **图像超分辨率计算模块：**在 PYNQ-Z2 的 Zynq-7020 平台上，ARM 处理系统负责图片、视频和 USB 摄像头输入，可编程逻辑实现邻域构建、四维查表、五顶点单纯形插值及方向融合。PS 与 PL 通过 AXI DMA 交换 RGBX 图像，支持图片和视频 x2/x4 放大，以及 480×270 摄像头输入的 x4 重建。x2 配置单个插值核，x4 配置三个同构核，以共享任务调度提高计算并行度。

2. **跨板缓存与显示模块：**PYNQ 将重建结果与操作界面合成为完整画面，经 HDMI 传输至 Nexys Video。Nexys 利用 DDR3 帧缓存衔接 1080p30 输入与 1080p60 DisplayPort 扫描，通过完整帧发布、缓冲所有权转移和延迟回收管理画面更新，使算法处理与显示扫描分别按照各自约束运行。

作品配备显示交互软件，提供并排比较、Wipe **滑动对照**、**ROI 局部**处理、视频逐帧播放及摄像头冻结保存，支持重建结果与最近邻基线比较、原始像素导出，便于观察局部细节并复核结果。

本系统核心亮点包括**查表存储与多核计算联合配置、整数插值末端统一舍入、跨板帧管理及 ARM 合成优化**。五组测试图集的 x4 平均 PSNR-Y 达到 29.631 dB，较双三次插值提高 0.998 dB，形成了集硬件重建、交互观察与结果保存于一体的端侧终端。

关键词：图像超分辨率；SR-LUT；FPGA；PYNQ；软硬件协同

目录

摘要 1
第一部分 作品概述 4
1.1 功能与特性 4
1.2 应用领域 4
1.3 主要技术特点 5
1.4 主要性能指标 5
1.5 主要创新点 6
1.6 设计流程 7
第二部分 系统组成及功能说明 8
2.1 整体介绍与方案选择 8
2.1.1 选题背景与设计目标 8
2.1.2 系统总体架构与数据通路 8
2.1.3 软硬件划分依据与模块接口关系 10
2.2 SR-LUT 超分辨率算法原理 10
2.2.1 查表模型与整体计算流程 10
2.2.2 邻域构建、量化与四维寻址 11
2.2.3 五顶点单纯形插值 12
2.2.4 方向恢复、融合与输出生成 12
2.3 硬件系统设计与实现 12
2.3.1 硬件平台、外设与板间连接 12
2.3.2 PYNQ 计算与显示硬件架构 13
2.3.3 SR-LUT 加速核与多核任务调度 14
2.3.4 PS/PL 数据交换与控制接口 14
2.3.5 Nexys 视频接收、帧缓存与显示输出 15
2.4 软件系统设计与实现 15
2.4.1 软件总体架构与主运行流程 15
2.4.2 图片、视频和摄像头输入管理 16
2.4.3 硬件调用、倍率切换与缓冲管理 16
2.4.4 显示布局、交互控制与 ROI 处理 16
2.4.5 图像评价、同帧冻结与结果保存 16
2.4.6 启动、停止与异常恢复 17
2.5 优化过程与设计取舍 17
2.5.1 LUT 采样间隔与存储容量选择 17
2.5.2 查表并行度与计算资源分配 17
2.5.3 ARM 图像合成与内存访问优化 18
2.5.4 缓冲管理与同倍率 Overlay 复用 19
第三部分 完成情况及性能参数 20
3.1 系统整体实物与运行状态 20

3.1.1 系统正面及斜 45° 全景 20
3.1.2 实屏运行效果 22
3.2 工程成果与功能展示 22
3.2.1 图片超分辨率与倍率切换 22
3.2.2 ROI 局部处理与 Wipe 对照 24
3.2.3 视频播放与逐帧观察 26
3.2.4 摄像头处理与同帧冻结保存 26
3.3 性能测试与结果分析 27
3.3.1 图像质量与局部细节对比 27
3.3.2 优化前后的性能与资源对比 33
3.3.3 FPGA 资源利用与时序结果 34
第四部分 大模型辅助工程开发记录 36
4.1 性能定位与 ARM 兼容性纠错 36
4.2 模式切换修复与自我复核 36
4.3 智能体工作流与作用 36
第五部分 总结 37
5.1 可扩展之处 37
5.2 心得体会 37
第六部分 PYNQ Skill 提炼与验证 38
6.1 从失败与观察中提炼技能包 38
6.2 有效性验证、适用边界与迁移 40
第七部分 参考文献 41
附录 A 关键接口与寄存器 42
附录 B 工程复现与运行方法 43
B.1 硬件、软件环境与工程准备 43
B.2 源码构建与产物检查 44
B.3 部署、接线与演示运行 45
B.4 结果核对与正常退出 46
B.5 Skill 离线运行与迁移入口 46

第一部分 作品概述

1.1 功能与特性

系统将低分辨率图像处理、细节观察和结果保存组合为统一操作界面。图片模式支持 x2/x4 放大、图片切换、任意区域框选查看与 PNG 保存；视频模式支持同步对比播放、暂停、逐帧、速度和循环控制；摄像头模式将 480×270 输入放大为 1920×1080，支持冻结、恢复及同帧快照。

图 1-1 展示三类输入的功能组织。**SR full、Compare、Nearest 和 Wipe** 四种布局分别用于重构画面查看、前后并排比较、最近邻基线对比和滑动分界观察；Fit、1:1 与平移用于整体及局部查看，三种模式均同步显示质量指标。

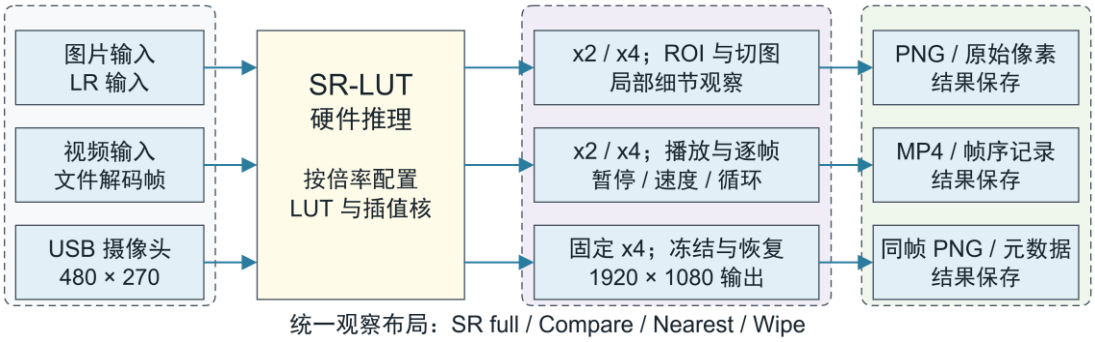


图 1-1 三类输入与交互功能

1.2 应用领域

本系统面向低分辨率画面（图像/视频/实时摄像头拍摄）的画质提升。如图 1-2 所示，典型应用包括电路图查看、监控图像分析和灾害救援现场观察等。电路图含有密集引脚、细线和文字，ROI 与滑动对照可辅助检查局部结构；监控视频可通过暂停、逐帧及局部放大观察远距离目标；救援现场图像可在受限传输条件下先保留低分辨率输入，再由终端完成重建与显示。针对不同场景的画质需求，可提供针对性的图像清晰度提升，极大提高了画面观感和判断的准确性。



图 1-2 电路图查看、监控分析与灾害救援应用场景

1.3 主要技术特点

算法采用 SR-LUT 固定查表方法[1,2]，将在线推理转化为四维寻址、五顶点插值与方向融合。x2 使用一个插值核；x4 通过较稀疏 LUT 和三个同构核兼顾存储与并行度。数值通路使用有符号整数运算，在四方向融合末端统一舍入和饱和。

系统根据数据处理特征划分软硬件职责：PS 管理输入和交互，PL 执行查表、插值与融合，Nexys 生成独立显示时序。ARM 合成采用整像素操作与缓存复用；PYNQ 和 Nexys 分别管理各自显示缓冲，在完整帧边界更新画面。

1.4 主要性能指标

系统主要性能指标见表 1-1。图像质量、软件优化及资源时序的测试结果在 3.3 节进一步分析。

表 1-1 系统主要性能指标

类别	参数项	指标/数值	备注
计算重建 PYNQ-Z2	重建倍率与算法	x2 / x4；SR-LUT Model S	图片、视频可切换倍率；摄像头固定 x4
	计算核心与时钟	x2 单核；x4 三核；100 MHz	通道与方向任务按可用核心分配
	数值与传输格式	RGB 各 8 位；int8 LUT；32 位 RGBX	有符号整数插值，融合末端统一舍入
	x4 查找表容量	单核 104,976 B；三核 314,928 B	q=32；单核原始表较 q=16 缩减 92.14%
	x4 完整系统资源	19,220 LUT；18,035 FF；113 BRAM36 等效块；2 DSP	布局布线结果，LUT 占用 36.13%
	x4 实现时序	WNS 0.144 ns；WHS 0.014 ns	建立与保持时序失败端点均为 0

类别	参数项	指标/数值	备注
缓存显示 Nexys Video	板间视频输入	HDMI 1920×1080，30 Hz	PYNQ 输出完整界面帧
	显示输出	DisplayPort 1920×1080，60 Hz	按显示时序扫描当前完整帧
	帧缓存组织	DDR3 三帧缓存	完整帧发布，旧帧引用退出后回收
	DP 显示桥资源	16,039 LUT；34,469 FF；25 BRAM36 等效块；0 DSP	WNS 0.106 ns；WHS 0.039 ns
系统功能与性能	输入与摄像头尺寸	图片、视频、USB 摄像头；480×270 → 1920×1080	尺寸对应摄像头 x4 重建
	Set5 x2 平均画质	PSNR-Y 35.510 dB；SSIM-Y 0.9477	q=16；PSNR-Y 较 OpenCV cubic 提高 1.543 dB
	Set5 x4 平均画质	PSNR-Y 29.631 dB；SSIM-Y 0.8425	q=32；PSNR-Y 较 OpenCV cubic 提高 0.998 dB
	板端像素一致性	已保存样本与固定参考模型逐像素一致	含 x2、x4 及 ROI 输出，详见表 3-3
	ARM 合成耗时	中位数 42.735 ms	较旧版本的 175.771 ms 降低 75.69%
	显示确认新帧率	22.6143 fps	窗口内确认 463 帧，时长 20.474 s
	观察与交互	SR full、Compare、Nearest、Wipe；ROI、Fit、1:1 与平移	支持整图、局部及同位置对照
	结果保存与回看	PNG 保存、视频 MP4 导出、暂停逐帧、摄像头冻结	冻结快照保留同帧输入与重建结果

1.5 主要创新点

（1）查表采样与多核调度联合配置。将 x4 的 LUT 采样间隔由 q=16 调整为 q=32，并配置三个同构插值核共享通道与方向任务。单核原始表容量缩减 92.14%，使三份 LUT 能够部署于片上，同时将可保留的未完成任务由 2 个增至 6 个。

（2）有符号插值与融合末端统一舍入。设计包含 7 位权重、15 位插值分子和 17 位融合结果的有符号整数通路，保留负权重分支，在四方向累加后统一舍入与饱和，避免逐方向取整引入额外误差。

（3）跨板完整帧发布与延迟回收。在 PYNQ 与 Nexys 两侧独立管理三帧缓存，以完整帧事件转移缓冲所有权，并在总线、预取与扫描引用全部退出后回收旧帧。该机制使 HDMI 30 Hz 输入与 DP 60 Hz 扫描独立运行，避免发布未完成帧或提前覆盖仍在使用的缓冲。

（4）RGBX 整字合成与静态布局复用。将逐通道背景广播改为 RGBX 整像素填充，融合像素检查与复制，并缓存静态界面布局，减少 ARM 对整帧数据的重复访问。在相同摄像头 x4 配置的短窗口测试中，合成耗时中位数由 175.771 ms 降至 42.735 ms，降低 75.69%，显示确认新帧率由 12.0794 fps 提高至 22.6143 fps。

1.6 设计流程

设计依次完成参考模型固定、整数算法分解、RTL 实现、DMA 联调和双板显示，随后集成交互功能，并依据执行时间和资源占用进行优化。图 1-3 给出各阶段的验证产物，上一阶段经验证的输出作为下一阶段的测试依据。

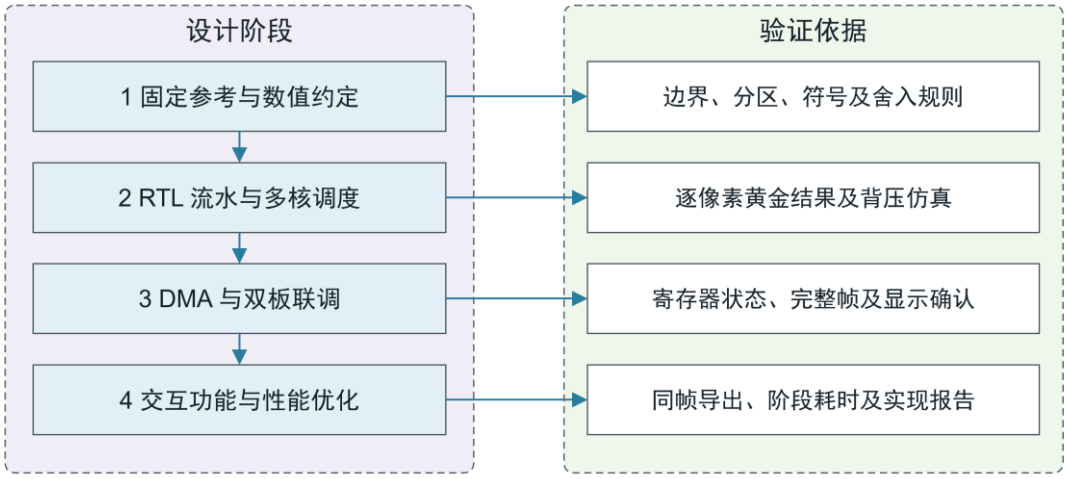


图 1-3 设计流程与阶段产物

第二部分 系统组成及功能说明

2.1 整体介绍与方案选择

2.1.1 选题背景与设计目标

超分辨率重建需要从低分辨率输入重构高分辨率像素。最近邻插值直接复制像素，计算简单但块状边界明显；双三次插值依据固定邻域权重平滑过渡，无法针对训练数据学习局部映射。SR-LUT 将训练模型在离散输入格点上的输出预先保存，推理时用查表和插值逼近模型映射[1]，为资源受限的 FPGA 提供了可分解的计算形式。

本项目采用 Model S 及固定 LUT，将研究重点放在**整数算法一致性、查表存储组织、软硬件协同与交互显示**。四维输入使存储量随格点数的四次方增长，因而采样间隔和并行核数需要共同选择；高分辨率结果的搬运与合成又要求从完整数据流评价性能。设计目标是在 PYNQ-Z2 上实现可重复的 x2/x4 推理，并支持图片、视频和摄像头输入，以及重建结果的显示、比较和保存。

2.1.2 系统总体架构与数据通路

如图 2-1 所示，PYNQ-Z2 承担计算与界面交互任务，Nexys Video 负责视频接收和输出。主数据流依次经过输入准备、AXI DMA、SR-LUT 核心、ARM 合成、HDMI 传输、Nexys DDR3 帧缓存和 DP 显示。超分结果首先返回 PS 内存，再与基线、指标及交互元素组合为完整界面帧。

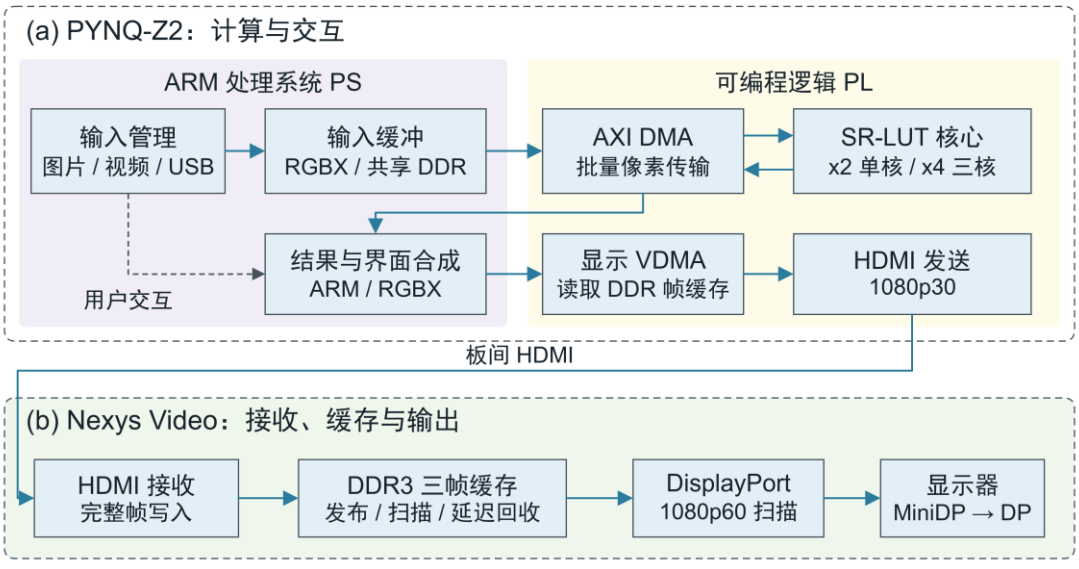


图 2-1 双板总体架构与显示接入方式

（1）输入管理与交互控制模块。该模块对应图 2-1 中 PS 侧的输入管理与用户交互部分，由 ARM 端输入程序、模式调度和事件处理组成。图片按素材选择读取源图，视频根据播放位置解码取帧，USB 摄像头持续采集并提供当前画面。三类输入经过尺寸整理和格式转换，共用后续计算与显示通路；其中，图片和视频支持 ×2/×4 重建，摄像头以 480×270 图像进入固定 ×4 通路。

摄像头与鼠标通过 USB 集线器接入 PYNQ。鼠标、板上按键和拨码开关产生的操作用于选择输入模式、倍率、ROI 及显示布局。源图、倍率或 ROI 改变时，软件更新处理任务，将选定像素组织为共享 DDR 中的 RGBX 输入缓冲，并向计算模块提供宽高和倍率等参数。输入调度由此将采集、用户操作与硬件调用衔接起来。

（2）超分重建与画面合成模块。该模块由 ARM 控制程序、共享 DDR、AXI DMA 和 PL 内的 SR-LUT 核心共同组成，对应图中的输入缓冲、算法核心及结果合成部分。ARM 准备输入后，AXI DMA 将 RGBX 像素送入 PL；核心依次完成邻域构建、量化寻址、五顶点查表插值和四方向融合，再将输出像素块重排为完整图像。×2 配置使用单个插值核，×4 配置使用三个同构核，调度器按可用核分派通道与方向任务，并按任务顺序提交结果。

重建结果经 DMA 写回共享 DDR 后，由 ARM 生成最近邻对照并组织显示内容。合成程序将源图与超分结果布置为并排 Compare、滑动 Wipe 或单结果画面，叠加操作控件与状态信息；图片具备配对参考图时，同时展示 PSNR-Y、SSIM-Y 等指标。最终形成可供显示通路读取的 RGBX 界面帧。结果保存功能从对应输入与重建结果中导出像素，使画面观察与结果文件关联到同一次处理。

（3）跨板传输与缓存显示模块。该模块连接图 2-1 中 PYNQ 的显示 VDMA、HDMI 发送和 Nexys 的接收、缓存与输出通路。PYNQ 将合成画面写入显示缓冲，由 VDMA 读取并按 1920×1080、30 Hz 的 HDMI 时序发送。Nexys 接收视频并检查有效区域，将完整帧写入板载 DDR3，再由 DisplayPort 通路按 1920×1080、60 Hz 扫描输出。两板之间传送的是已经包含对照布局和交互元素的完整画面。

PYNQ 与 Nexys 分别管理各自的三帧缓冲，协调画面写入、等待显示和扫描读取。新帧完整写入后才具备发布条件，显示端在安全边界切换画面，旧缓冲在扫描及相关访问结束后回收。没有新结果时，输出端继续扫描当前完整帧，因此显示刷新按固定时序运行，画面更新则跟随处理结果到达。

2.1.3 软硬件划分依据与模块接口关系

划分依据是任务的数据依赖和控制复杂度。文件访问、摄像头采集、鼠标事件及文件编码具有较多状态，适合由 Linux 和 ARM 管理；邻域计算、寻址及插值的数据关系固定，适合在 PL 中形成流水线。显示桥独立实现接收、缓存与扫描，使显示调试不依赖算法内部状态。表 2-1 列出各模块的输入输出及接口。

表 2-1 模块职责与接口

模块	主要职责	输入与输出接口
PYNQ PS	源帧准备、交互、结果合成与保存	USB/文件 → RGBX；AXI-Lite 控制；共享 DDR
PYNQ PL	SR-LUT 推理、DMA 搬运、HDMI 扫描	AXI4-Stream 输入/输出；HDMI
Nexys Video	完整帧接收、缓存和 DP 输出	HDMI → DDR3 → MiniDP
主机	部署、启动、配置与状态读取	以太网、JTAG、UART

其中，AXI-Lite 传递启动与尺寸参数，AXI4-Stream 传递像素，DDR 缓冲承接 PS 与 PL 的批量交换

2.2 SR-LUT 超分辨率算法原理

2.2.1 查表模型与整体计算流程

SR-LUT 的离线阶段在采样格点上保存模型输出，在线阶段对输入邻域进行量化、寻址和插值。工程使用固定的 int8 查表值；倍率为 s 时，每个表项输出一个 s×s 子像素块。RGB 三个通道分别计算，结果重新组合为彩色图像。图 2-2 给出从输入邻域到输出光栅的完整过程。

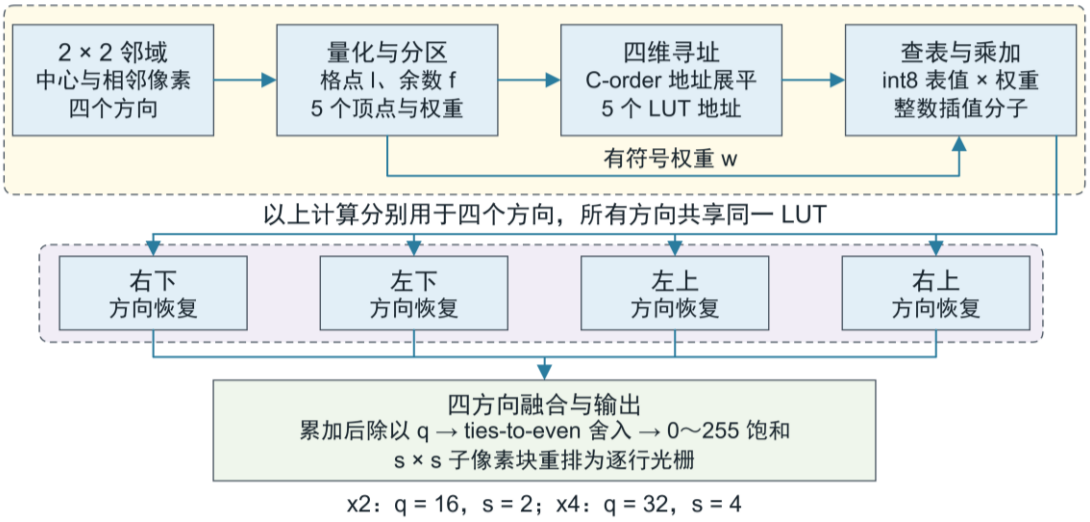


图 2-2 SR-LUT 查表与方向融合流程

每个中心像素沿四个方向形成邻域，插值结果恢复空间方向后累加。实现保留整数插值分子，直到方向融合完成才执行统一除法和舍入，从而避免逐方向取整引入额外偏差。

2.2.2 邻域构建、量化与四维寻址

对单通道图像，中心像素及相邻三个像素组成四维输入 p=(a,b,c,d)。四个方向依次为右下、左下、左上和右上，图像边界使用 reflect 扩展。设采样间隔为 q，每维格点数为 L，像素分解见式（2-1）。

$$l_i = \lfloor \frac{p_i}{q} \rfloor,\ f_i = p_i - q l_i,\ L = \frac{256}{q} + 1 \tag{2-1}$$

式中，l_i 为低位格点坐标，f_i 为余数，i 对应 a、b、c、d 四个维度。x2 配置 q=16、L=17；x4 配置 q=32、L=9。由于 q 为 2 的整数次幂，量化可以用移位和位选完成。上边界保留额外格点，以支持最大像素值所在区间的插值。

四维格点按 C-order 展平。顶点掩码 m 的四个分量分别对应 a、b、c、d，取值为 0 或 1，查表地址如式（2-2）。

$$A(m) = (l_a + m_a)L^3 + (l_b + m_b)L^2 + (l_c + m_c)L + l_d + m_d \tag{2-2}$$

每个地址返回 4 个（x2）或 16 个（x4）有符号表值。地址展平顺序与 ROM 初始化文件一致，因此可以用同一组中间量核对参考实现、转换脚本和 RTL。

2.2.3 五顶点单纯形插值

四维单纯形插值根据四个余数的比较关系选择五个顶点。以标准降序分区 $f\pi_1 \geq f\pi_2 \geq f\pi_3 \geq f\pi_4$ 为例，五个权重依次为 $q-f\pi_1$、$f\pi_1-f\pi_2$、$f\pi_2-f\pi_3$、$f\pi_3-f\pi_4$、$f\pi_4$，权重和为 q。对输出子像素 k，整数插值分子按式（2-3）计算。

$$N_k = \sum_{j=0}^{4} w_j \, T[A(m_j), k] \tag{2-3}$$

式中，T 为 LUT，m_j 为第 j 个顶点掩码，w_j 为对应权重，k 的范围为 0 至 s^2-1。工程逐项保持固定参考实现的比较分支和相等值处理；兼容分支的实际权重包含负值，x2 范围为 −13～16，x4 为 −29～32。**权重、乘积和累加通路均按有符号数处理**。

2.2.4 方向恢复、融合与输出生成

四个方向使用同一 LUT，其输出小块需先恢复到原图坐标系，再进行融合。令 $N^{(r)}$ 为方向 r 的插值分子，R^{-r} 表示对应的逆方向变换，输出规则如式（2-4）。

$$Y = \mathrm{clip}_{[0,255]}\left(\mathrm{round}_{\mathrm{even}}\left(\frac{\sum_{r=0}^{3} R^{-r}(N^{(r)})}{q}\right)\right) \tag{2-4}$$

式中 $\mathrm{round}_{\mathrm{even}}$ 为四舍六入五成双舍入，clip 将结果限制在 0～255。当前模型的四方向累加结果只除以 q。单方向分子使用 15 位，四方向融合使用 17 位；x4 已核对的范围分别为 −11,491～11,459 和 −45,964～45,836，均在相应位宽内。

计算产生的 s×s 小块按图像坐标重排，形成逐行输出光栅。内部小块 FIFO 与输出 FIFO 缓冲各流水级的瞬时吞吐差异；当下游撤销 ready 时，尚未接收的数据及其位置状态保持不变，握手完成后再更新。

2.3 硬件系统设计与实现

2.3.1 硬件平台、外设与板间连接

计算平台 PYNQ-Z2 使用 Zynq-7020，将双核 ARM 处理系统与可编程逻辑集成于同一器件[3]；显示平台 Nexys Video 使用 Artix-7 XC7A200T，板载 DDR3 用于整帧缓存[4]。系统采用开发板现有接口连接，无额外 PCB。表 2-2 给出连接关系。

表 2-2 硬件连接与信号用途

连接	用途	配置
PYNQ USB → 集线器	摄像头、鼠标接入	供电 USB 集线器
PYNQ HDMI OUT → Nexys HDMI IN	传输完整交互画面	1920×1080，30 Hz
Nexys MiniDP → 显示器 DP	主要显示出口	1920×1080，60 Hz
Nexys MiniDP → VGA 转换器	替代显示接入	独立 VGA 配置
主机 → PYNQ / Nexys	部署、下载与状态采集	以太网 / JTAG / UART

主机用于部署和控制，图像处理及输出在两块板上完成。UART 为状态观测通道，JTAG 用于 Nexys 配置；两者与视频像素通路分离，便于定位输入、处理和显示各阶段的问题。

2.3.2 PYNQ 计算与显示硬件架构

PYNQ PL 包含 SR-LUT 核心、AXI DMA、显示 VDMA 和视频时序控制模块。计算通路使用 100 MHz 时钟，HDMI 像素时钟为 74.25 MHz。AXI DMA 的 MM2S 通道连接 PS HP0，S2MM 通过 ACP 返回结果；VDMA 经 HP1 读取合成后的 RGBX 图像

x2 与 x4 分别生成匹配的 BIT/HWH 文件，配置中绑定倍率、LUT 和核数量。HWH 提供软件查找 IP、寄存器与地址映射所需的信息，运行时按倍率选择整套 Overlay。图 2-3 直观展开算法核心中的缓存、任务流水和插值核。

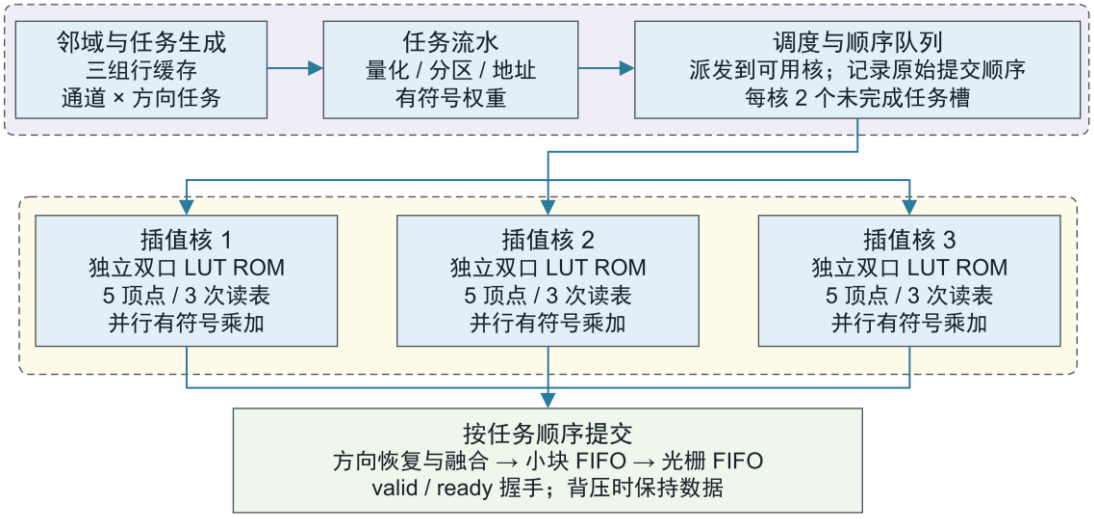


图 2-3 SR-LUT 加速核心与同构插值核

2.3.3 SR-LUT 加速核与多核任务调度

输入进入三组行缓存后，邻域生成模块形成通道和方向任务。任务流水依次完成量化、比较分区及地址生成，再由调度器送入可用插值核。每个核拥有独立双口 LUT ROM，五个顶点分三次读取；一个表项中的 s^2 个值并行参加乘加。

x2 使用一个核，x4 使用三个核。**三个核共享通道与方向任务，不固定绑定 RGB 通道**。每核保留两个未完成任务槽，顺序队列深度为核数的两倍，记录通道、方向、核编号和结束标志。调度器允许读表、计算与提交交叠，并按任务原序返回结果，使后级方向融合保持确定顺序。

图 2-3 中，核组输出经过方向恢复、融合、小块 FIFO 和光栅 FIFO。各级用 valid/ready 传递背压；任务入口、内部计算和输出重排共同遵循这一约束，避免下游暂停时覆盖尚未消费的结果。

2.3.4 PS/PL 数据交换与控制接口

图像采用 32 位 AXI4-Stream，经 AXI DMA 在共享内存和算法核心之间传输[5]。每个字包含一个 RGBX 像素：[7:0] 为 R，[15:8] 为 G，[23:16] 为 B，[31:24] 的 X 字节为零。TKEEP 固定为 1111，TLAST 标记整帧最后一个像素。输入 H×W 个字，输出 $H \times W \times s^2$ 个字。

480×270 输入对应 518,400 B，x4 输出对应 8,294,400 B。软件先检查尺寸和缓冲类型，完成输入同步，再提交 S2MM 接收、配置并启动核心、提交 MM2S 发送；两

路完成后核对状态和结果。提前准备接收可为核心输出提供可用通道，缓存同步则依照实际端口的一致性约定执行。

AXI-Lite 管理启动、宽高、状态、错误码与周期计数。数据通路通过流握手执行，软件通过状态寄存器确认完成。寄存器地址及语义列于附录 A。

2.3.5 Nexys 视频接收、帧缓存与显示输出

PYNQ 的显示 VDMA[6]采用三块 1920×1080×4 B 缓冲，分别承担合成写入、待显示和扫描持有。Nexys 经 HDMI 接收画面，检查有效区域与光栅几何，再通过跨时钟 FIFO 和 MIG DDR3 通路[7]写入独立三帧缓存。显示端复用 FPGA_DisplayPort 与 Digilent 视频接口模块[8,9]，在此基础上实现完整帧发布、运行状态监测和板间协同控制。

两级缓存分别维护缓冲所有权。PYNQ 的显示缓冲由合成写入转为待显示，VDMA 确认切换后才回收旧扫描缓冲；Nexys 帧槽依次经过 FREE、WRITING、READY、PENDING、SCANNING 和 RETIRING 状态。只有完整有效帧可以发布，输出端在安全扫描边界选取新帧，旧槽在 AXI 事务、预取 FIFO 与扫描引用全部退出后回收。

没有空槽时，接收端丢弃新输入整帧，保留正在扫描的数据；没有新帧时，输出端继续扫描当前帧。输入中断通过代次标识区分新旧事务，多位状态采用原子跨时钟传递。

2.4 软件系统设计与实现

2.4.1 软件总体架构与主运行流程

板端软件分为输入管理、硬件执行、结果组织和显示交互。Runtime 接收事件并选择源帧，Engine 管理 Overlay 与计算会话，FramePair 绑定输入、输出和帧身份，Compositor 生成界面，Scanout 管理提交及显示缓冲。一轮处理依次经过输入调度、缓冲同步与 DMA 调用、结果确认、界面合成和显示提交；冻结、保存与退出通过对应事件进入各自处理分支。

每次主循环迭代（tick）先处理按键、拨码和鼠标事件，再决定是否需要新的算法计算。新源帧、倍率或 ROI 变化触发处理；仅改变对照布局、平移或工具栏时复用已有结果。硬件操作由所属线程串行执行，文件写入和质量评分通过后台任务完成，以减少耗时操作对交互循环的影响。

2.4.2 图片、视频和摄像头输入管理

图片使用配对低分辨率与高分辨率素材，在切图、倍率或选区改变时更新任务。视频以播放位置和帧序号管理输入，播放、暂停、逐帧和循环改变取帧策略；暂停时可反复观察同一结果。摄像头采集进程持续更新最新帧，处理线程读取当前可用帧，统一为 480×270 后送入 x4 处理通路。

三类输入在进入硬件前统一为 RGBX 数据，并附带来源、代次与序号。结果保存与后台评分使用这些身份字段关联请求，避免迟到结果覆盖已切换的图像或选区。

2.4.3 硬件调用、倍率切换与缓冲管理

Engine 根据目标倍率选择匹配的 Overlay，创建输入与接收缓冲，按 2.3.4 节的顺序调用 DMA。x2/x4 切换需要更换硬件配置；当前 r17 软件在倍率不变、仅显示缓冲属性变化时复用 Overlay，并重建必要显示资源和恢复 DMA 通道，以减少相同计算配置的重复加载。

缓冲分为算法输入、算法输出与显示画面三类。算法输出只有在 DMA 完成后交给合成模块，显示缓冲只有在确认退出扫描后才能再次写入。摄像头处理采用两块结果缓冲轮换，当前计算与上一结果的合成提交由会话顺序协调。

2.4.4 显示布局、交互控制与 ROI 处理

Compare 并排显示最近邻与 SR 结果，Wipe 在同一图像坐标上以可移动分界线切换两种结果，适合观察同位置边缘差异。SR full 和 Nearest 提供单结果显示。

ROI 处理先把用户选区向外扩展一像素上下文，再按硬件尺寸约束建立处理窗口；结果按倍率换算坐标，裁回原选区。内部边缘使用源图中的真实邻域，原图边界执行 reflect 扩展。

2.4.5 图像评价、同帧冻结与结果保存

图片质量任务绑定图像、倍率、ROI 与代次，切换素材时可实时更新指标。PNG 保存包含输入、最近邻、SR、配对参考图及 export.json，文件中的像素与展示缩放分离。摄像头 Freeze 复制当前输入与结果形成独立快照，冻结后可改变布局或保存，Resume 恢复实时输入。

视频软件设有逐帧超分与 MP4 编码入口，按源帧序列组织输出，并记录帧序号与像素哈希。功能展示采用已有视频界面素材，完整导出操作与结果核对方法见附录 B。

2.4.6 启动、停止与异常恢复

主机启动器完成本地包与设备检查、Nexys 配置、板端会话启动和状态读取。每轮会话保存运行身份和恢复记录，使源程序、显示配置与相关服务能够对应到同一次运行。

正常停止按输入任务、后台工作、算法 DMA、显示 DMA、缓冲释放和服务恢复的顺序收尾。出现异常时，先结合当前会话日志和阶段状态定位故障，再执行对应恢复流程；图像采集、算法完成和显示确认分别记录，便于判断停滞发生位置。

2.5 优化过程与设计取舍

2.5.1 LUT 采样间隔与存储容量选择

固定 LUT 的原始容量由每维格点数与每项输出大小共同决定。8 位表值下，单核容量见式（2-5），其中 s 为放大倍率。

$$M = L^4 s^2 = (256/q + 1)^4 s^2 \text{ B} \tag{2-5}$$

x4 若取 q=16，单核原始表值就需要 1,336,336 B，超过目标器件的 BRAM 容量；q=32 将深度降为 6,561，单核仅需 104,976 B，为多核复制留出空间。图 2-4 展示存储与画质的关系：较大采样间隔扩大插值区间，平均画质下降几乎可忽略不计，但显著减少存储占用。

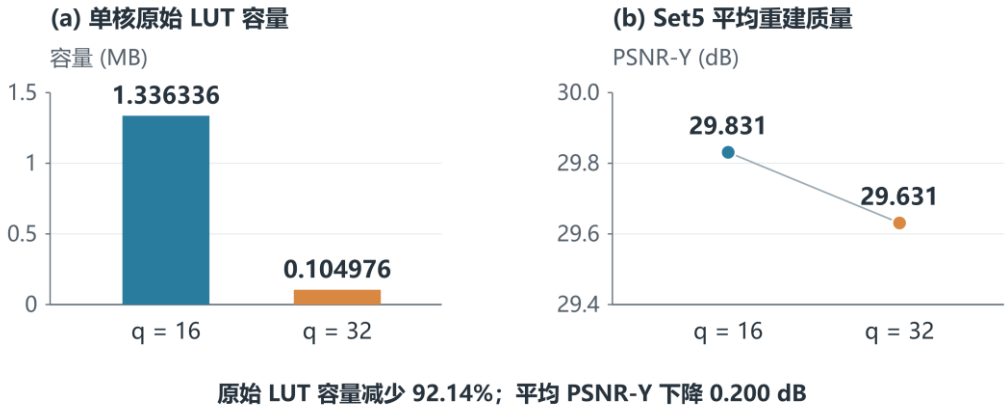


图 2-4 x4 采样间隔的存储与质量取舍

2.5.2 查表并行度与计算资源分配

在 q=32 的 x4 配置下，单核只能在一组 ROM 与乘加通路中调度任务。增加同构插值核后，可交叠处理不同通道和方向的任务。表 2-3 比较相同 LUT 配置下的单核基线结构与三核采用结构。

表 2-3 同一 x4 配置下的并行结构优化

结构指标	单核结构	三核结构
同构插值核数	1	3
每核未完成任务槽	2	2
总任务槽 / 顺序队列深度	2	6
独立双口 LUT ROM	1 组	3 组
原始查找表总容量 / B	104,976	314,928

三核配置将总任务槽从 2 个增加到 6 个，以三份 LUT 存储换取更多任务重叠机会；调度器按可用核心分配任务，并按原序提交结果。最终 x4 完整系统占用 113 个 BRAM36 等效块，处于器件 140 块的容量范围内。

2.5.3 ARM 图像合成与内存访问优化

一帧 1080p RGBX 图像约为 8.29 MB。r10 中的逐通道背景填充会多次遍历整帧，延长 ARM 合成及鼠标事件处理的等待时间。r11 将其改为 uint32 整像素写入，结合 X 字节检查与复制融合、静态布局复用，减少完整图像的重复访问；两版使用相同 FPGA 位流。

图 2-5 对比摄像头 x4、Compare/Fit、DP 直连条件下的历史观测结果。合成耗时中位数由 175.771 ms 降至 42.735 ms，降低 75.69%；相应窗口的显示确认帧率由 12.0794 fps 提高至 22.6143 fps，表明 ARM 合成是此次优化的主要收益来源。

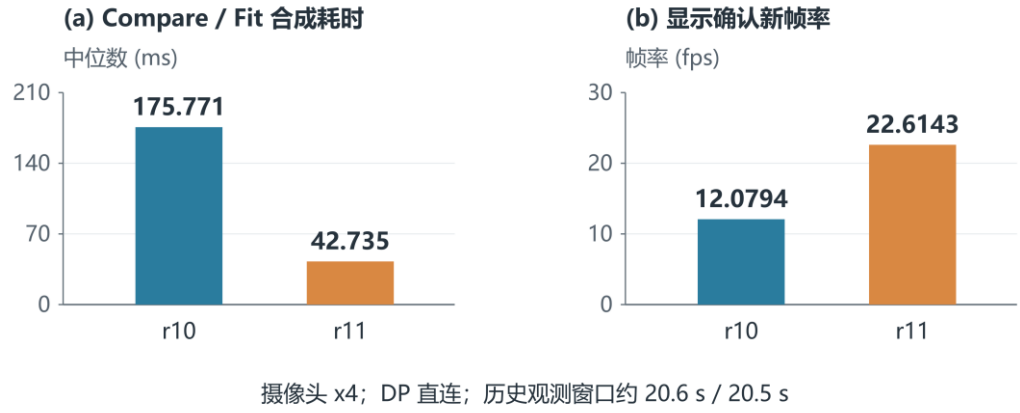


图 2-5 ARM 合成优化前后的耗时与显示确认帧率

2.5.4 缓冲管理与同倍率 Overlay 复用

图片、视频与摄像头模式的显示缓存策略不同。原实现将缓存策略变化与算法配置变化一并触发 Overlay 加载，使同为 x4 的模式切换重复配置 FPGA。优化后分别判断倍率、故障状态与缓存策略：算法配置保持有效时保留 Overlay，只重建必要缓冲，并显式恢复 DMA 双向通道。表 2-4 给出固定切换序列的前后结果。

表 2-4 同倍率模式切换的配置复用效果

同倍率切换步骤	修改前	修改后
首次进入 x4 图片模式	加载 x4 Overlay	加载 x4 Overlay
x4 图片 → 摄像头	因缓存策略变化再次加载	保留 Overlay，重建必要缓冲
摄像头 → x4 视频	再次加载 x4 Overlay	保留 Overlay，重建必要缓冲
上述序列累计加载次数	3 次	1 次
切换后的 DMA 恢复	随 Overlay 初始化	显式启动接收和发送通道

上述序列的 Overlay 加载次数由 3 次减为 1 次，减少两次重复配置。回归检查同时验证同倍率往返时配置身份保持不变、DMA 通道重新启动；x4 切换至 x2 时仍加载对应位流。显示层继续利用结果缓冲轮换和完整帧发布，将算法配置复用与帧缓存回收分别管理。

第三部分 完成情况及性能参数

3.1 系统整体实物与运行状态

3.1.1 系统正面及斜 45° 全景

系统由 PYNQ-Z2、Nexys Video、摄像头、鼠标、显示器和连接线构成。图 3-1 展示双板、摄像头与鼠标的整体实物，图 3-2 给出双板及两块开发板的斜 45° 视图，可辨认板卡布局、HDMI 板间连接及外设接口。

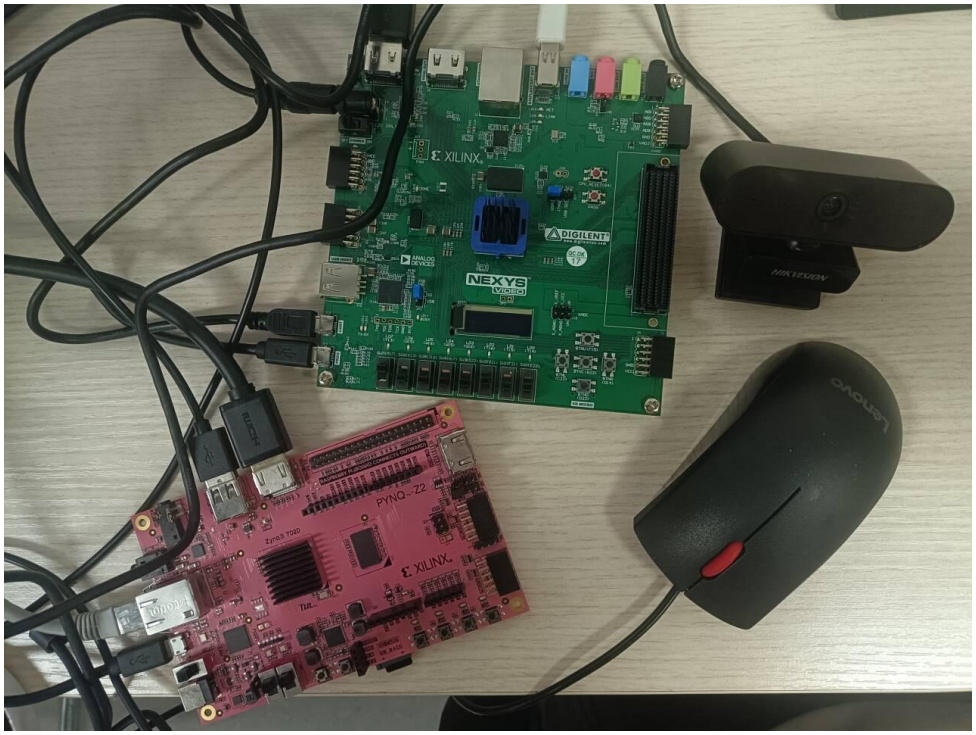
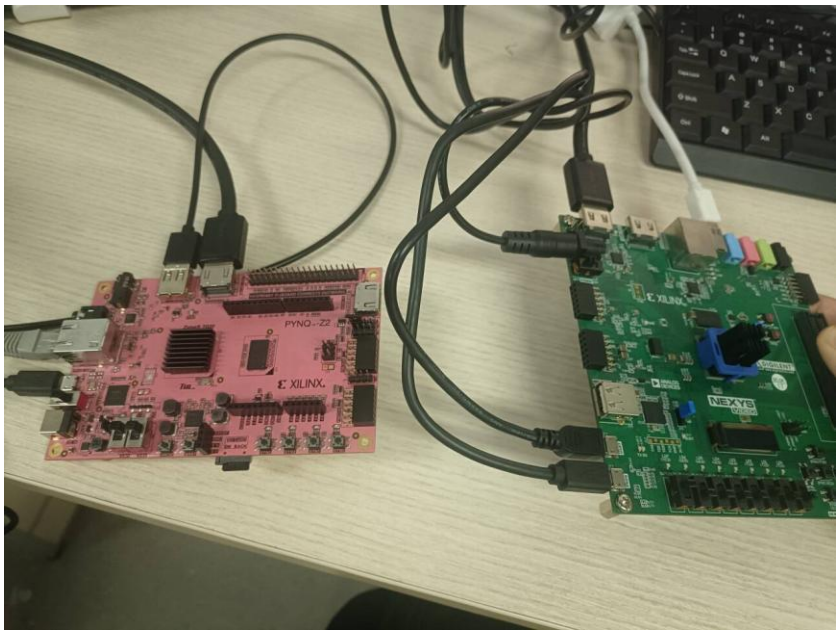


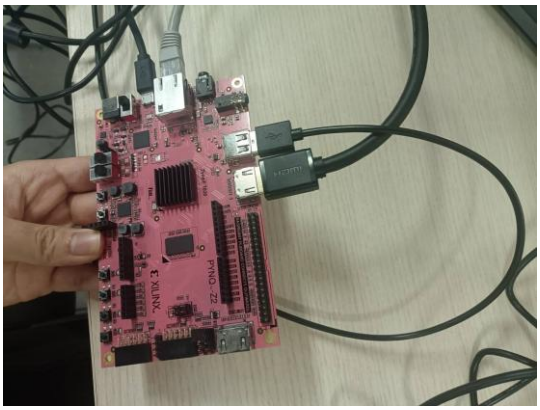
图 3-1 系统整体实物图



(a) 双板连接



(b) Nexys Video



(c) PYNQ-Z2

图 3-2 双板及单板斜 45° 视图

3.1.2 实屏运行效果

图 3-3 用于展示显示器上的实际运行画面，画面应同时包含显示器边框和可辨认的交互界面，便于对应系统连接与输出状态。

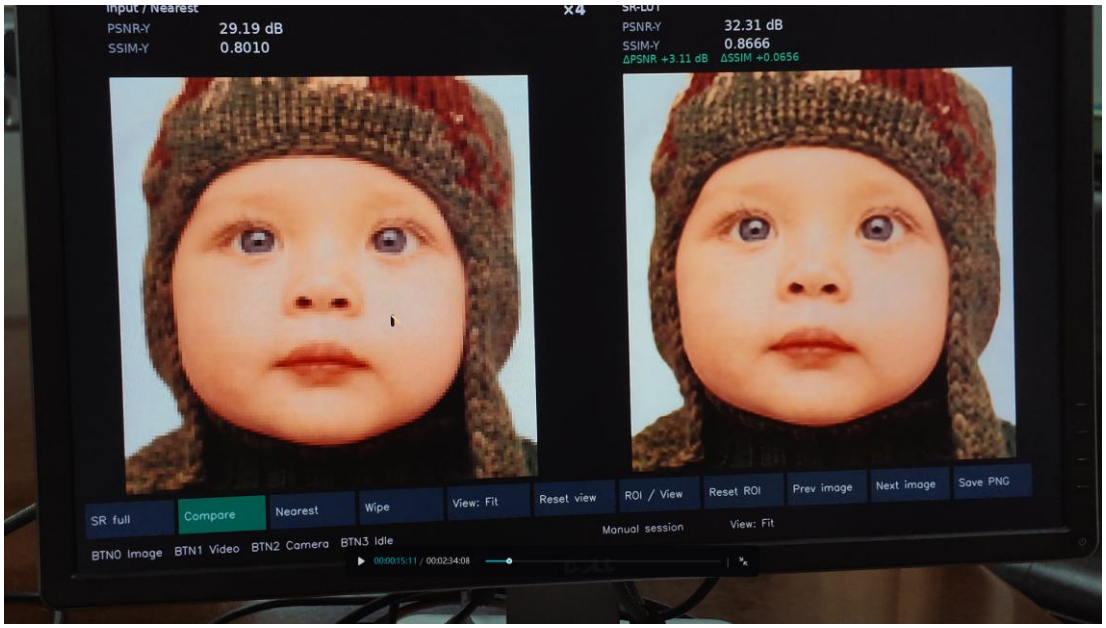


图 3-3 实屏运行效果

3.2 工程成果与功能展示

3.2.1 图片超分辨率与倍率切换

图片处理流程包括原生低分辨率输入、FPGA 推理、对照显示与 PNG 保存。图 3-4 和图 3-5 分别展示 butterfly 与 woman 的 x4 处理结果。图中同位置裁剪便于比较轮廓及细线的变化。

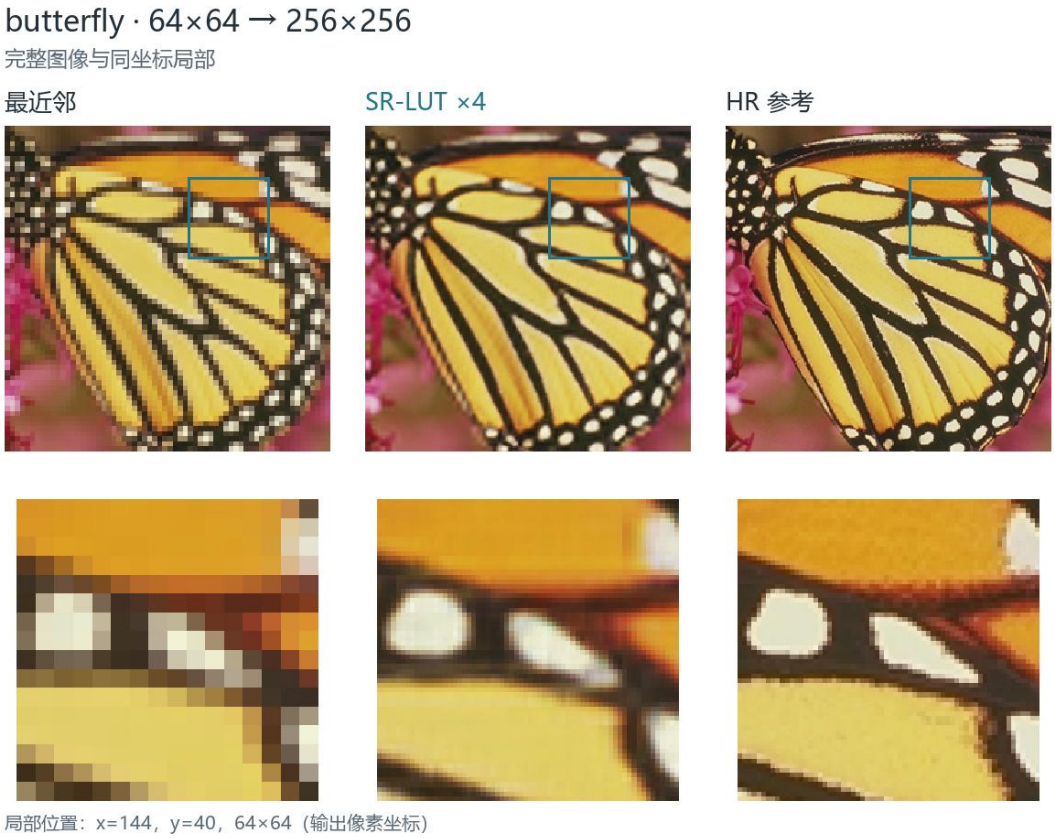


图 3-4 butterfly 的 x4 板端处理结果

butterfly 的低分辨率输入为 64×64，输出为 256×256。翅膀斜向轮廓在最近邻图中呈阶梯状，SR 结果的边缘轮廓更连续；细纹区域的变化在局部放大中更容易观察。



图 3-5 woman 的 x4 板端处理结果

woman 的 x4 输入为 57×86，输出为 228×344。图 3-5 中帽沿与面部轮廓可用于观察边缘过渡；对同一参考图切换至原生 x2 输入后，可比较倍率变化对细节恢复的影响。

3.2.2 ROI 局部处理与 Wipe 对照

图 3-6 展示 woman 的任意区域框选（ROI） x4 结果与原生 x2 处理。ROI 选区为 (5,16,48,27)，保存的目标区域为 192×108；x2 全图输入为 114×172，输出为 228×344。选区计算保留邻域上下文，同时记录选区位置与倍率。



图 3-6 ROI 局部处理与 x2 图像结果

Wipe 布局把两种结果放在相同坐标上，拖动分界线即可查看同一轮廓从最近邻到 SR 的变化。图 3-7 由同帧板端导出像素经程序离线合成，左侧为原始图像，右侧为板端重构图像。

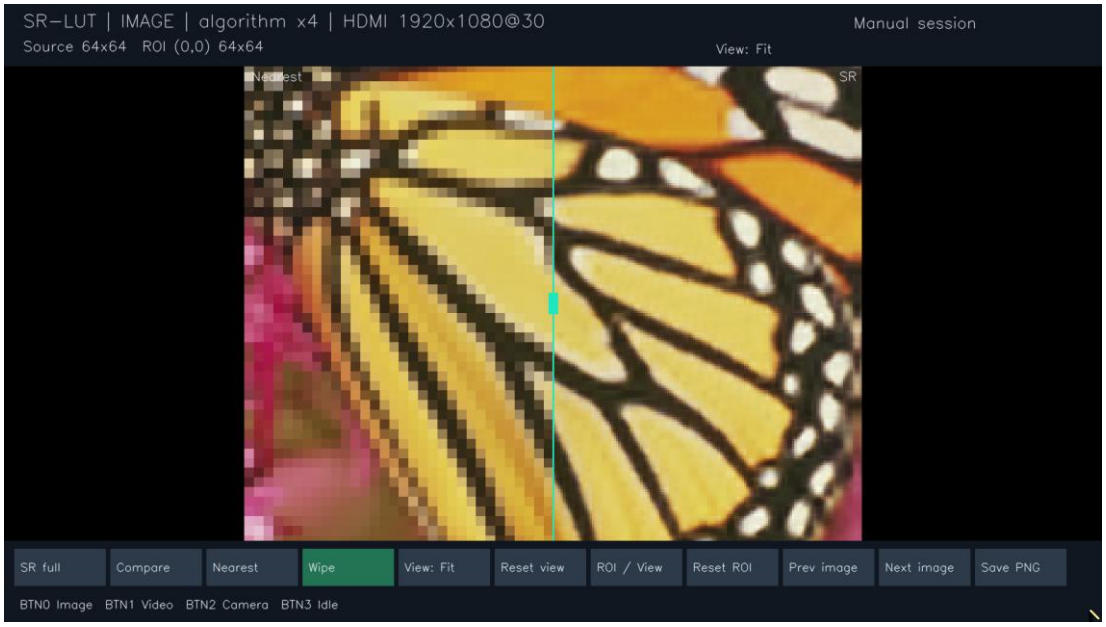


图 3-7 图片 Wipe 滑动对照界面

3.2.3 视频播放与逐帧观察

视频控制包括播放、暂停、前后逐帧、速度和循环，倍率选择对应 x2 或 x4 计算配置。图 3-8 为程序的视频界面离线预览，顶部提供模式与布局控制，底部显示播放位置与进度。

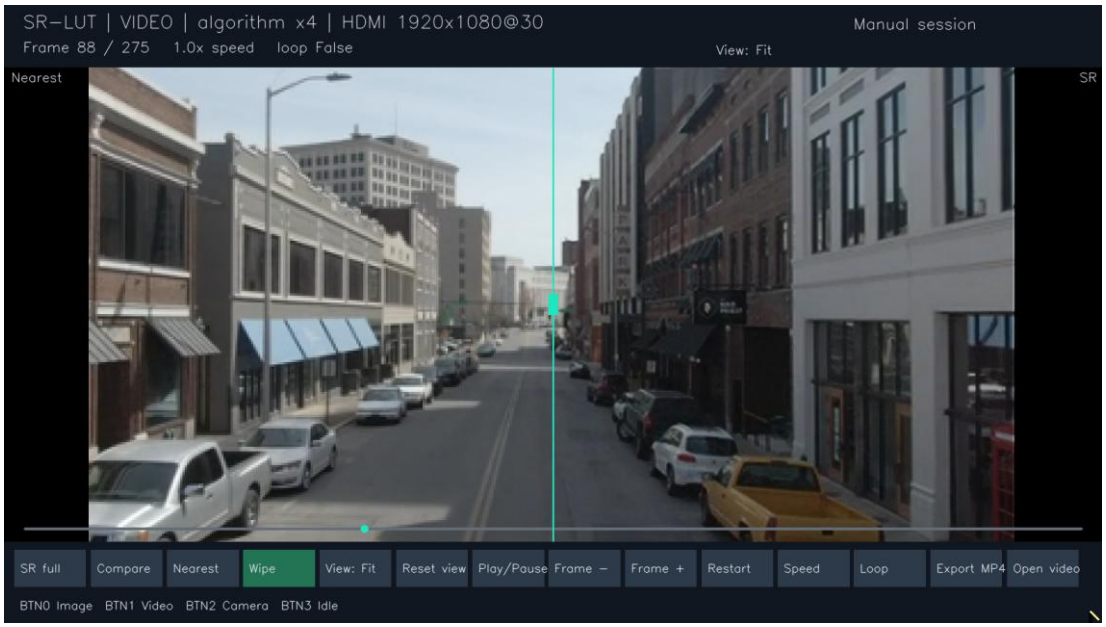


图 3-8 视频播放控制与 Wipe 对照界面

暂停与逐帧操作可以固定目标帧，结合 Wipe 查看建筑边缘或道路细节；速度控制改变播放调度节奏，可循环播放。

3.2.4 摄像头处理与同帧冻结保存

摄像头模式固定采用 480×270 输入与 x4 输出。图 3-9 选用实时摄像头拍摄后冻结保存的同帧结果，展示对比效果。



图 3-9 摄像头同帧输入与超分结果

冻结后，用户可切换视图并保存。图中的文字轮廓和细线用于观察局部变化；恢复操作重新接收实时输入。该流程将现场画面保存为输入与输出一一对应的图像数据，供后续质量分析与结果复核。

3.3 性能测试与结果分析

3.3.1 图像质量与局部细节对比

质量评价采用 Set5[10] 的 baby、bird、butterfly、head 和 woman 五组原生 LR/HR 图集，使用固定 SR-LUT 参考模型输出，与最近邻和 OpenCV 4.10.0 INTER_CUBIC 比较。PSNR-Y 在 RGB 转 Y 后裁去四像素边界，峰值取 255；SSIM-Y[11] 在完整 Y 图上使用 11×11、σ=1.5 的高斯窗口进行有效卷积，沿用工程保存的 LeRF 评价实现[12]。各图分别计算后取算术平均，结果见表 3-1。

表 3-1 Set5 平均图像质量

倍率	方法	PSNR-Y / dB	SSIM-Y
x2	Nearest	30.837	0.9002
x2	OpenCV cubic	33.967	0.9330
x2	SR-LUT，q=16	35.510	0.9477
x4	Nearest	26.258	0.7383
x4	OpenCV cubic	28.633	0.8137
x4	SR-LUT，q=16	29.831	0.8484
x4	SR-LUT，q=32	29.631	0.8425

图 3-10 对比了不同方法的平均画质。实际 x4 配置 q=32 的平均 PSNR-Y 比最近邻高 3.373 dB，比 OpenCV cubic 高 0.998 dB；x2 配置相对 cubic 提高 1.543 dB。

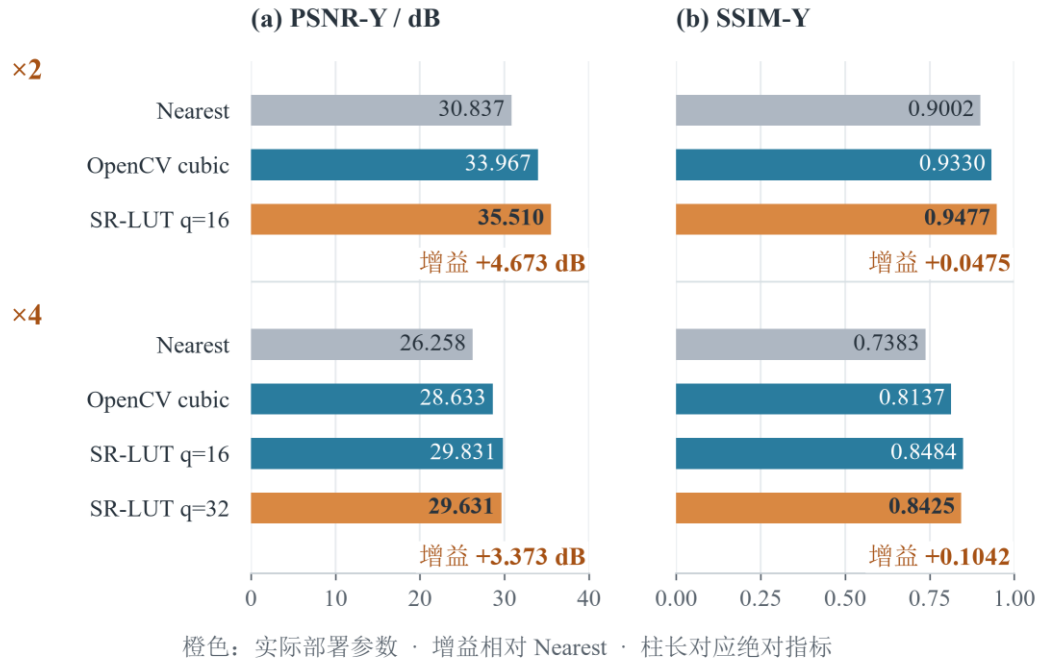


图 3-10 Set5 平均图像质量与实际部署配置的增益

表 3-2 进一步展开 x4 逐图结果，比较不同图像内容及两种采样配置下的重建质量差异。

表 3-2 x4 逐图质量比较

图像	OpenCV cubic	SR-LUT q=16	SR-LUT q=32
baby	32.018 / 0.8627	32.513 / 0.8713	32.309 / 0.8666
bird	30.437 / 0.8764	31.603 / 0.8939	31.324 / 0.8878
butterfly	22.320 / 0.7353	24.858 / 0.8398	24.756 / 0.8351
head	31.709 / 0.7594	32.002 / 0.7689	31.724 / 0.7598
woman	26.682 / 0.8348	28.179 / 0.8679	28.042 / 0.8631

图 3-11 将表 3-2 的逐图结果转换为相对 cubic 的画质增益。q=32 在 butterfly 和 woman 上的 PSNR-Y 分别提高 2.436 dB 和 1.360 dB，在 baby、bird 和 head 上分别提高 0.291 dB、0.887 dB 和 0.015 dB，体现出图像内容对重建收益的影响。

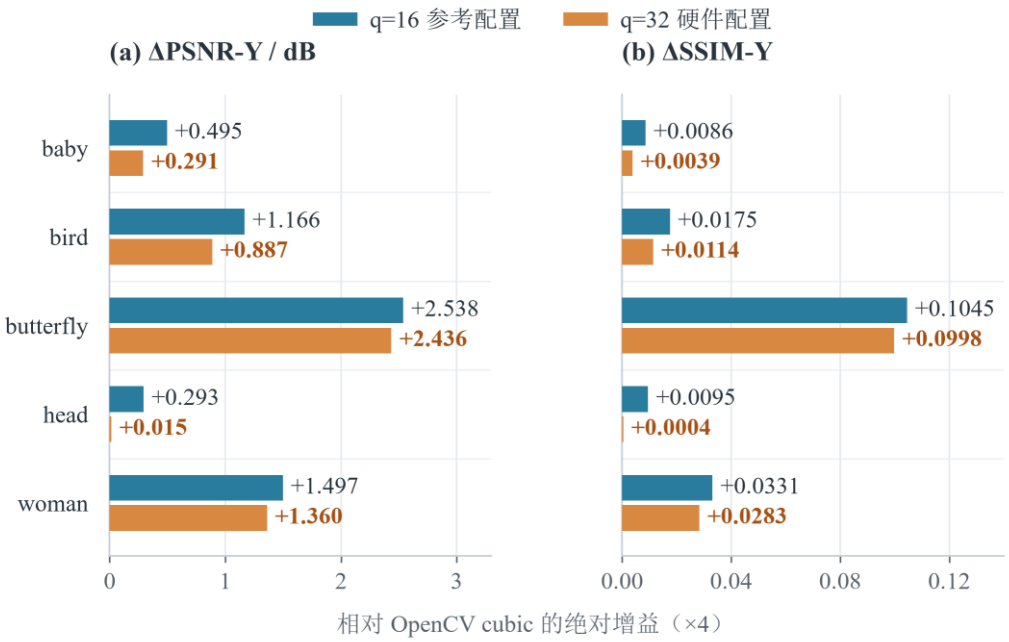


图 3-11 五组图像相对 OpenCV cubic 的 x4 画质增益

图 3-12 至图 3-16 展示五组图像的局部细节：上方标记 HR 选区，下方依次比较 HR、最近邻、双三次插值与两种 LUT 采样配置，所有局部均按相同坐标裁剪。

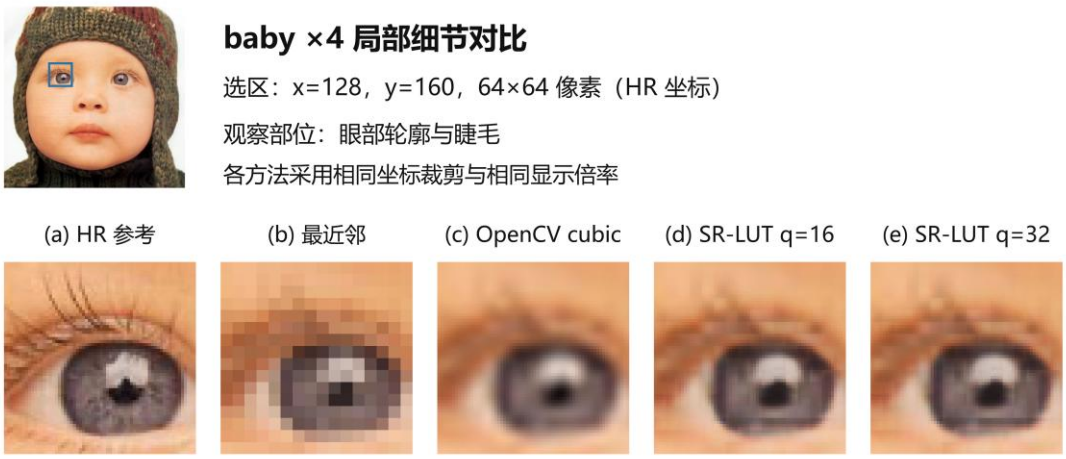


图 3-12 baby 的 x4 方法与局部细节比较

图 3-12 中，最近邻在虹膜与眼睑边缘形成明显像素块，双三次插值的过渡较平滑；SR-LUT 对眼睑轮廓的过渡作了进一步调整。

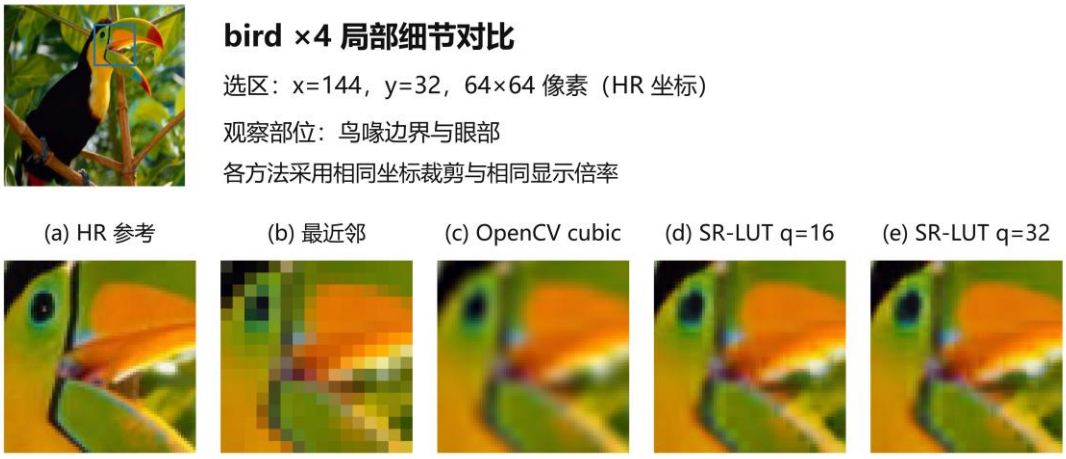


图 3-13 bird 的 x4 方法与局部细节比较

图 3-13 比较鸟头与喙部的轮廓。SR-LUT 改善了最近邻的阶梯边界，并在眼部和喙部保留较清楚的明暗分界；两种采样配置的局部差异可结合表 3-2 的逐图指标分析。

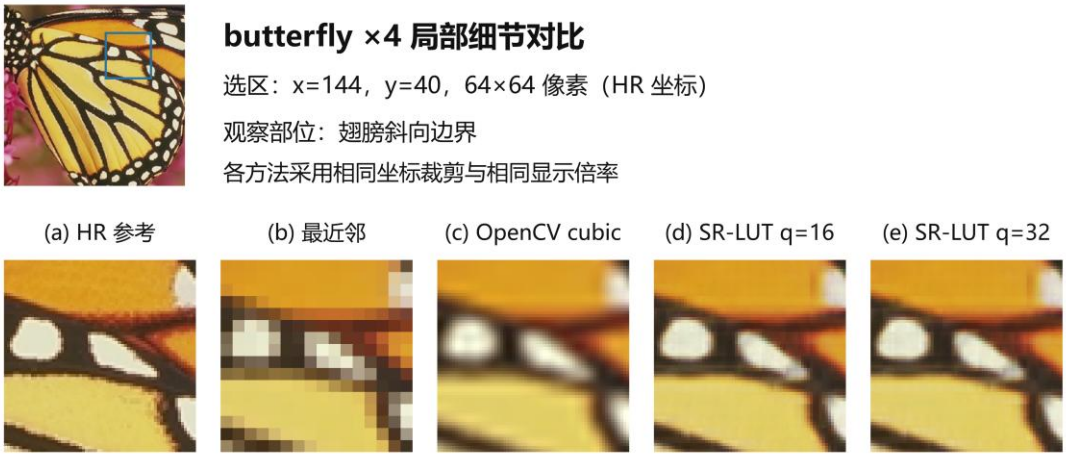


图 3-14 butterfly 的 x4 方法与局部细节比较

图 3-14 中，翅膀斜线和亮色斑纹构成明显边界。SR-LUT 的轮廓连接比最近邻连续，相较双三次插值也保留了更鲜明的边缘，与该图较大的 PSNR-Y 增益相对应。

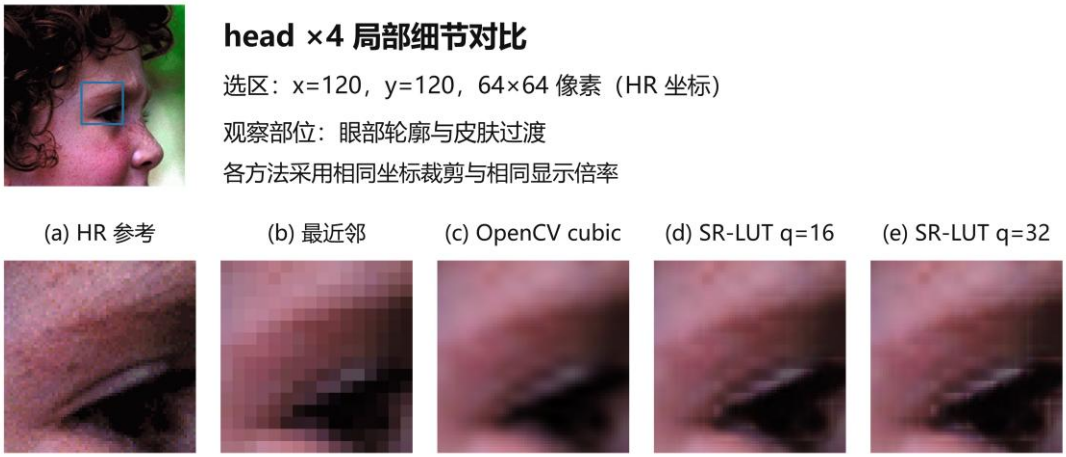


图 3-15 head 的 x4 方法与局部细节比较

图 3-15 展示眼部轮廓及皮肤纹理。各插值方法均平滑了原图中的细小起伏，q=32 与双三次插值的整体评分接近；该样本表明，查表重建对不同图像结构的改善程度存在差异。



图 3-16 woman 的 x4 方法与局部细节比较

图 3-16 中，SR-LUT 的眼部轮廓比最近邻连贯，边缘过渡比双三次插值更集中。q=16 与 q=32 的比较同时呈现了采样间隔变化对局部质量的影响，为片上存储与重建质量的取舍提供直观依据。

表 3-3 给出板端保存图像的质量指标。各样本的 SR 输出与固定参考模型逐像素一致，验证了对应输入下板端实现与参考模型的数值一致性。

表 3-3 板端保存图像的质量结果

板端样本	输入 → 输出尺寸	PSNR-Y / dB	SSIM-Y
butterfly x4	64×64 → 256×256	24.756	0.8351
woman x4	57×86 → 228×344	28.042	0.8631
woman ROI x4	48×27 → 192×108	29.395	0.8989
woman x2	114×172 → 228×344	34.171	0.9618

3.3.2 优化前后的性能与资源对比

查表配置优化的主要收益是存储缩减。如图 3-17 所示，x4 的采样间隔由 q=16 调整为 q=32 后，单核原始 LUT 从 1,336,336 B 减至 104,976 B，减少 92.14%。每维格点数由 17 降至 9，总地址数由 83,521 降至 6,561，每个表项仍包含 16 个 int8 输出。

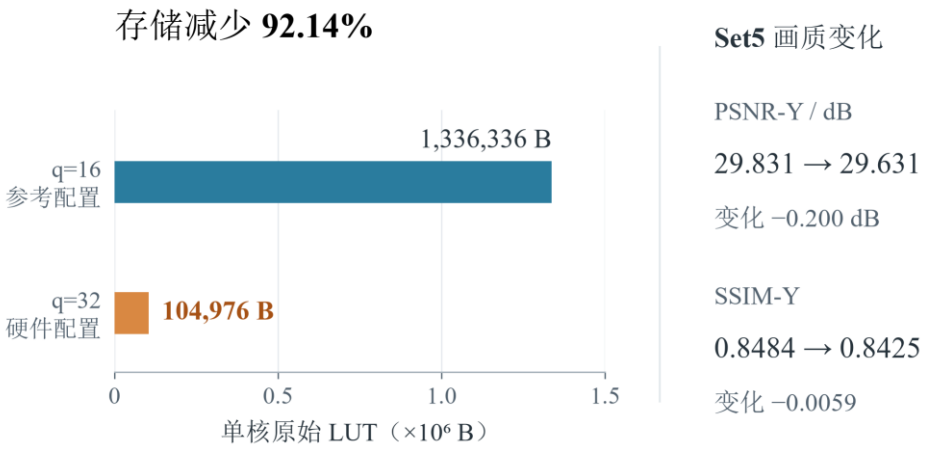


图 3-17 x4 查表配置的存储与画质取舍

这一取舍使 Set5 平均 PSNR-Y 从 29.831 dB 降至 29.631 dB，SSIM-Y 从 0.8484 降至 0.8425。q=16 用于软件参考比较，q=32 用于 x4 三核实现；后者以较小的画质变化换取显著的存储缩减，使多核实现满足目标器件的片上存储容量约束。

软件优化针对合成与数据搬运。图 3-18 显示，合成耗时中位数由 175.771 ms 降至 42.735 ms，降低 75.69%；显示确认帧率由 12.0794 fps 提高至 22.6143 fps，提高 87.21%，约为优化前的 1.87 倍。

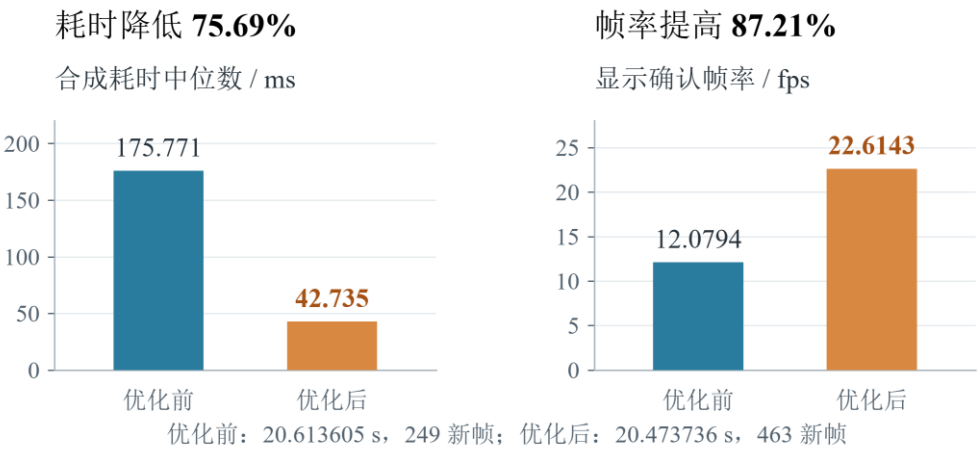


图 3-18 ARM 渲染优化前后的短窗口结果

图 3-18 的帧率按各自短窗口内确认的新帧数与窗口时长计算。本轮优化修改了像素处理、合成和调度相关软件，PYNQ x2/x4 与 Nexys 的硬件配置字节一致。表 3-4 给出相应资源对照，软件性能提升未增加 FPGA 资源占用。

表 3-4 软件优化前后的 FPGA 资源对照

配置	LUT 前 / 后	FF 前 / 后	BRAM36 等效前 / 后	DSP 前 / 后
PYNQ x2	10,724 / 10,724	12,580 / 12,580	105 / 105	3 / 3
PYNQ x4	19,220 / 19,220	18,035 / 18,035	113 / 113	2 / 2
Nexys DP	16,039 / 16,039	34,469 / 34,469	25 / 25	0 / 0

3.3.3 FPGA 资源利用与时序结果

表 3-5 汇总 Vivado 2026.1 布局布线后的资源。算法 IP 为 PYNQ 完整系统内部层级，包含加速通路及其接口包装；Nexys 分别列出主要 DP 配置和 VGA 转接配置。

表 3-5 布局布线后的 FPGA 资源占用

配置与范围	LUT	FF	RAMB36	RAMB18	DSP
PYNQ x2 完整系统	10,724	12,580	103	4	3
其中 算法 IP	4,343	1,976	98	2	3
PYNQ x4 完整系统	19,220	18,035	111	4	2
其中 算法 IP	12,831	7,431	106	2	2
Nexys DP 显示桥	16,039	34,469	24	2	0
Nexys VGA 显示桥	16,048	34,471	24	2	0

图 3-19 进一步展示 PYNQ 完整系统内部的资源组成。每根柱的总量由算法 IP 与其余系统组成，BRAM36 等效量按 RAMB36＋RAMB18/2 计算。PYNQ x4 完整系统的 LUT 占用为 19,220，约占 53,200 个可用 LUT 的 36.13%。

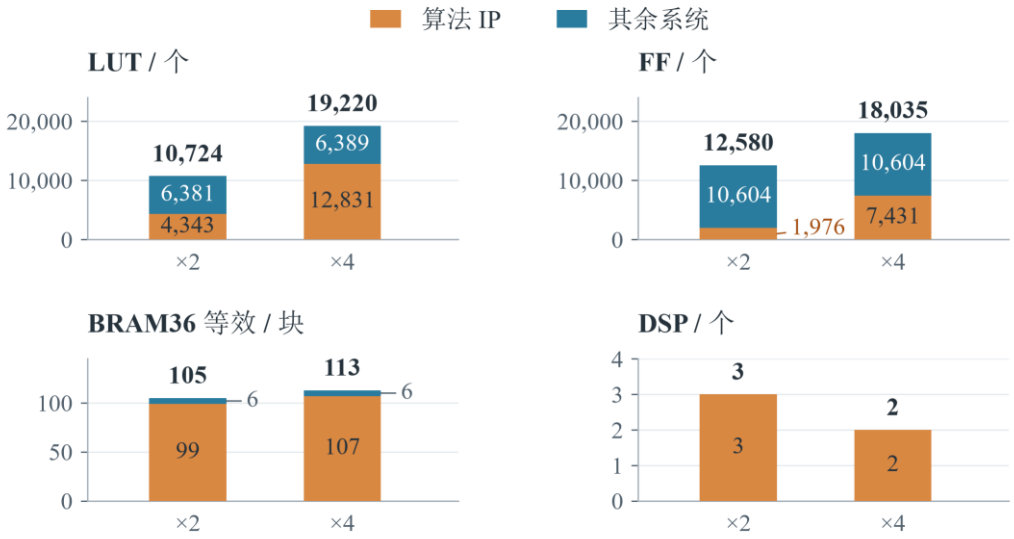


图 3-19 PYNQ 完整系统的 FPGA 资源组成

x4 的资源增加主要集中在算法 IP：其 LUT 数由 4,343 增至 12,831，BRAM36 等效量由 99 增至 107；其余系统的 FF 数均为 10,604，BRAM36 等效量均为 6。Nexys 的完整帧主体存于板载 DDR3，片上 RAM 主要用于接口和流水缓存。

表 3-6 列出各完整系统的建立与保持时序结果。WNS 和 WHS 分别表示最差建立裕量和最差保持裕量，所有列示配置的失败端点均为零。

表 3-6 完整系统时序摘要

配置	Setup WNS / ns	Hold WHS / ns	失败端点 setup / hold
PYNQ x2	0.194	0.028	0 / 0
PYNQ x4	0.144	0.014	0 / 0
Nexys DP	0.106	0.039	0 / 0
Nexys VGA	0.106	0.039	0 / 0

PYNQ 计算时钟为 100 MHz，HDMI 像素时钟为 74.25 MHz；Nexys 含接收、DDR 与 DP 时钟域。结果说明当前布局布线满足相应约束。

第四部分 大模型辅助工程开发记录

我们将大语言模型用于工程中的性能分析、代码审查与故障排查。以下以 ARM 合成优化和模式切换修复为例，记录提示词、模型建议、实施纠错及验证结果。

4.1 性能定位与 ARM 兼容性纠错

优化目标与模型建议。在不改变 FPGA 位流、LUT 和算法分辨率的前提下，改善合成耗时和鼠标响应。模型辅助分析 ARM 合成路径，定位逐通道背景填充的内存访问开销，提出以 uint32 整像素写入替代通道广播，并复用静态布局。

实施纠错与效果。初版改动在 ARM 的 NumPy 1.21 环境中有 5 项测试失败，原因是非连续视图不能直接重解释。保留原行步长、改为逐行处理后，28 项 ARM 检查全部通过。优化后的合成耗时中位数由 175.771 ms 降至 42.735 ms，下降 75.69%，实屏反馈确认鼠标流畅度恢复。

4.2 模式切换修复与自我复核

问题与提示词。联调时出现菜单隐藏后文字残留、摄像头切换异常。我们向 AI 模型提供实屏照片，提出“隐藏底部菜单栏时还是会出现”，并要求修复后“不会影响其他功能，比如鼠标灵敏度等”。

模型回答与方案。模型发现 Manual session 文字在显隐判断之外绘制，同时指出：“进入摄像头模式时，仅因显示缓存策略变化，就会重新加载同一份 FPGA 配置。”据此将文字与背景统一纳入工具栏显隐控制，并将 Overlay 加载与显示缓冲重建分开。

自我复核与成果。模型随后检查板端驱动，发现保留 Overlay 后还需重启已停止的 DMA 通道，补充双向通道启动及调用次数断言。主机回归中 248 项通过，最终菜单与模式切换相关 ARM 检查 24 项通过。同倍率图片、摄像头、视频的切换序列中，配置加载次数由 3 次减至 1 次，减少了重复配置操作。

4.3 智能体工作流与作用

智能体按“读取现象与约束—追踪源码和日志—局部修改—回归检查—实屏复核”执行。测试失败后，将错误信息带回同一任务继续修正，并保存代码差异和复测记录。团队负责现场观察与设计取舍，模型辅助定位、修改和验证。

第五部分 总结

本项目完成了 SR-LUT 整数推理、图片与视频及摄像头输入、ARM 合成和双板交互显示。采样间隔与三核配置满足片上存储约束，软件优化减少重复内存访问；质量测试、逐像素对照及实现报告验证了方案的可行性。

5.1 可扩展之处

后续完善倍率切换时评分任务与缓冲释放的同步，加强长时间运行的恢复能力；继续优化 PS 合成与数据搬运，并将画质评价扩展至复杂纹理和真实场景。Skill 将补充独立算子位流，并在更多 PYNQ 平台验证迁移过程。

5.2 心得体会

这三个月完成该项目的过程，对我们整个队伍而言是一次宝贵的工程学习经历，让我们收获了很多珍贵的设计经验，提高了工程思维。

本项目使我们认识到，图像算法在嵌入式平台上的实现，不只是将计算公式转换为 RTL。从参考模型到双板连续处理与显示，还需要解决数值表达、接口传输、缓存管理和软件交互问题。明确的接口约定和分阶段验证，是各模块协同运行的基础。

项目初期，参考算法与整数硬件实现的一致性是首要难点。寻址顺序、相等余数分区、表值符号和融合舍入均会影响最终像素。我们保存量化、顶点、权重、地址与插值分子并逐级对照，使误差能够定位到具体环节，也形成了先明确数值规则、再设计流水线的习惯。

优化过程中，我们逐步认识到计算、存储与画质之间的相互制约。x4 查表深度随格点数四次方增长，须先调整采样间隔，才能在片上部署三核。系统联调又表明，ARM 图像填充和布局合成同样影响处理速度。因此，性能分析必须覆盖输入、计算、合成和显示的完整链路。

双板联调加深了我们对缓存生命周期的理解。算法完成、HDMI 接收完成与 DP 扫描结束属于不同事件，缓冲必须在全部引用退出后才能回收。完整帧发布、帧身份关联和延迟回收，使软件对象与硬件事务之间的关系更加明确。

大模型协作中，具体的现象、源码和测试结果，比宽泛提问更容易形成有效分析。我们将建议转化为局部修改，再通过参考结果、回归检查和实屏观察验证；失败记录则

成为下一轮纠错的依据。这一过程积累了软硬件开发经验，也使我们形成了以观测定位问题、以验证检验方案的工程方法。

这次的项目经历，让我们完成了从“产品使用”到“系统设计”的转变。我们不仅收获了扎实的 FPGA 开发技能，更学会了工程设计的思维模式，为我们未来的学习和科研道路奠定了基础。

第六部分 PYNQ Skill 提炼与验证

6.1 从失败与观察中提炼技能包

技能包将算法控制移入可替换 hooks，数据约定写入配置，通用会话保留 PYNQ Overlay、allocate 与 DMA 调用[13]。表 6-1 对应列出工程观察、复用规则及验证结果。

表 6-1 工程观察到复用规则的提炼与验证

失败或观察	提炼的复用规则	验证方法与结果
驱动绑定 SR 寄存器、倍率和 RGBX 几何	以接口配置和 hooks 分离算法控制与通用会话	阈值、增益两个非超分示例分别核对 1,536 和 36 个元素
缓存依赖同步；超时后 DMA 可能仍访存	双向同步；确认 Halted 后释放，停止失败保留缓冲	CPU/设备分离存储；停止失败无 free，后续 run 拒绝
60 Hz 扫描可能重复显示同一帧	按阶段、代次和固定窗口统计新完成帧身份	重复事件、跨阶段及跨代次用例均验证计数
ARM 非连续视图失败；NumPy 属性冲突	检查类型、步长与环境；故障卡保留触发、修正和复测	三种 NumPy 环境各通过 28 项回归

（1）接口契约与算法控制解耦。原驱动将 SR-LUT 寄存器、放大倍率及 RGBX 几何尺寸写入调用流程，换用其他算子时需要连同驱动修改。提炼时，将输入输出形状和数据类型写入配置，并据此确定传输字节数，将寄存器配置、启动及完成判断分别封装为 configure、start、done 钩子。通用会话只负责缓冲分配、同步和传输，并在提交前检查数据约定。阈值分割与饱和增益两个非超分算例复用了同一会话，验证算法替换无需改写公共传输流程。

（2）缓存可见性与传输顺序。针对 CPU 缓存与设备存储不同步的问题，模板固化“复制输入—输入同步—清理并同步输出—配置核心—提交接收—启动核心—发送输入—等待完成—回读同步”的顺序，先准备接收通道，再允许核心产生输出。模拟后端将

CPU 与设备存储分开，仅通过同步操作传递数据，使漏同步能够在结果比对中暴露。回读后返回独立副本，并检查连续批次复用缓冲时，上一批结果不会被覆盖。

（3）超时后的停止确认与资源回收。软件等待超时并不意味着 DMA 已停止访存，直接释放缓冲会破坏仍在进行的事务。模板先请求停止双向通道，在限定时间内确认 Halted 状态后才释放内存；停止未确认时，将会话置为隔离状态，保留缓冲的强引用并拒绝再次提交。故障注入分别模拟传输停滞与停止失败，检查正常停止路径先停机后释放，停止失败路径不释放、不复用，由此将缓冲所有权纳入异常处理流程。

（4）从兼容性失败提炼环境检查。在原 ARM/NumPy 1.21 环境中，相关非连续数组视图不能沿用原有方式进行数据类型重解释，需要保留行步长并逐行处理。技能包据此检查输入形状和类型，并将非连续输入纳入回归。首次离线测试的 23 项中有 7 项出错，进一步定位到 NumPy 2.x 的 device 属性冲突；调整模拟缓冲的属性命名后，补充边界用例，并在三个 NumPy 环境分别运行 28 项测试。该过程形成“保留失败日志—定位依赖差异—修正实现—在对应环境复测”的故障排查方法。

（5）按新完成帧统计处理性能。联调中观察到，60 Hz 显示扫描可能重复输出同一帧，因此技能包明确区分显示刷新率与有效处理帧率。统计脚本使用单调时钟和固定时间窗口，按处理阶段、会话代次及帧身份筛选事件，只计入成功完成且未重复的新结果。测试构造重复编号、错误阶段、跨代次、未完成及窗口边界事件，检查筛选和去重；窗口开始前已完成的帧，也不会因窗口内再次出现而被重复计数。

（6）将软硬件匹配检查前移。部署检查将 BIT/HWH 的 SHA-256、同目录同主名关系、所需 IP 和 DMA 模式纳入加载前置条件，通过后才下载 Overlay。传输完成时，除检查通道状态和核心完成标志，还核对输入输出字节数。测试注入错误哈希、缺少 IP、SG 模式以及“预期 16 字节、实际 8 字节”的输出，分别检验加载前拦截与提前 TLAST 的识别能力，使文件版本和帧长度问题能在对应环节定位。

交付包包含提示词工作流、调用模板、验证脚本及故障排查四类内容。README 逐项说明**适用场景、使用方法、已验证效果和失效条件**，并提供对应入口。

提示词工作流进一步固定了每类任务的输入与交付物：接口设计须给出形状、类型、端序、字节数及 TLAST/TKEEP 约定；参考实现须给出独立黄金结果、边界输入和损坏输出用例；性能分析须依据阶段耗时与帧身份，提出最小修改并在相同条件下复测；部署排障须核对 BIT/HWH、状态和同步路径，再判断停止与回收条件。故障卡则保留触发条件、错误日志、修正方法及复测入口，便于其他团队沿相同路径定位问题。

6.2 有效性验证、适用边界与迁移

32×48 uint8 阈值分割与 9×4 int16 饱和增益的标量参考与独立 NumPy 模拟逐元素一致，损坏输出均被检出。同步、超时、停止失败和副本保持纳入回归；NumPy 2.2.6、2.5.4、1.26.4 各通过 28 项，独立目录复跑验证了可移植性。

独立算例与数值边界。阈值分割使用 32×48 的 uint8 灰度图和阈值 128，其中 127 输出 0，128 与 129 输出 255，逐元素比较共 1,536 个结果。饱和增益使用 9×4 的 int16 数据和 3 倍增益，覆盖正负极值与饱和边界：10,922 输出 32,766，20,000 饱和为 32,767，−20,000 饱和为 −32,768，共核对 36 个结果。黄金参考采用标量逐项计算，模拟后端采用独立的 NumPy 向量实现；再故意修改一个输出元素，确认检查器能够报告失败。

验证记录与可复现交付。每个算例保存输入、期望输出、实际输出和机器可读的 result.json，同时生成 report.md，便于复查数值结果与运行条件。回归覆盖同步顺序、结果副本、超时停止、缓冲隔离、文件核验及帧事件统计，并在独立目录重新运行，检查技能包对原工程路径的依赖。历史板端记录说明提炼规则的问题来源，独立算例与故障注入检验公共流程的有效性，跨环境回归检查依赖兼容性，三类记录共同构成验证依据。

适用边界由后端接口与故障测试确定：当前模板面向完整 Overlay、simple-mode 双向 AXI DMA、固定批次和单线程独占；VDMA、SG/cyclic、多调用者及部分重配置需要扩展。停止失败测试表明缓冲必须保留，因此超时恢复要求能够读取状态并确认停止。上述验证覆盖离线逻辑，新算子位流、跨板缓存和性能须在目标板验证。

迁移至图像滤波、音频块处理或传感器批处理时，替换数据约定、独立参考、hooks 与配套 BIT/HWH，再按最小、边界和连续批次完成板端核对。操作命令列于附录 B。

第七部分 参考文献

[1] JO Y, KIM S J. Practical Single-Image Super-Resolution Using Look-Up Table[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2021: 691-700. https://openaccess.thecvf.com/content/CVPR2021/html/Jo_Practical_Single-Image_Super-Resolution_Using_Look-Up_Table_CVPR_2021_paper.html.

[2] JO Y. SR-LUT: Practical Single-Image Super-Resolution Using Look-Up Table[CP/OL]. [2026-10-30]. https://github.com/yhjo09/SR-LUT.

[3] TUL Corporation. PYNQ-Z2 Reference Manual: v1.1[EB/OL]. 2019[2026-10-30]. https://dpoauwgwqsy2x.cloudfront.net/Download/PYNQ_Z2_User_Manual_v1.1.pdf.

[4] Digilent Inc. Nexys Video FPGA Board Reference Manual: DOC 502-316, Rev. A[EB/OL]. 2020[2026-10-30]. https://digilent.com/reference/_media/reference/programmable-logic/nexys-video/nexys-video_rm.pdf.

[5] Advanced Micro Devices, Inc. AXI DMA LogiCORE IP Product Guide: PG021, v7.1[EB/OL]. [2026-10-30]. https://docs.amd.com/r/en-US/pg021_axi_dma.

[6] Advanced Micro Devices, Inc. AXI Video Direct Memory Access Product Guide: PG020, v6.3[EB/OL]. [2026-10-30]. https://docs.amd.com/r/en-US/pg020_axi_vdma.

[7] Advanced Micro Devices, Inc. Zynq 7000 SoC and 7 Series Devices Memory Interface Solutions User Guide: UG586[EB/OL]. [2026-10-30]. https://docs.amd.com/r/en-US/ug586_7Series_MIS.

[8] FIELD M. FPGA_DisplayPort: An Implementation of DisplayPort Protocol for FPGAs[CP/OL]. [2026-10-30]. https://github.com/hamsternz/FPGA_DisplayPort.

[9] Digilent Inc. Digilent Vivado Library[CP/OL]. [2026-10-30]. https://github.com/Digilent/vivado-library.

[10] BEVILACQUA M, ROUMY A, GUILLEMOT C, et al. Low-Complexity Single-Image Super-Resolution based on Nonnegative Neighbor Embedding[C]//Proceedings of the British Machine Vision Conference. BMVA Press, 2012: 135.1-135.10. DOI: 10.5244/C.26.135.

[11] WANG Z, BOVIK A C, SHEIKH H R, et al. Image Quality Assessment: From Error Visibility to Structural Similarity[J]. IEEE Transactions on Image Processing, 2004, 13(4): 600-612. DOI: 10.1109/TIP.2003.819861.

[12] LI J, et al. LeRF-PyTorch: Learning Resampling Function for Adaptive and Efficient Image Interpolation[CP/OL]. [2026-10-30]. https://github.com/ddlee-cn/LeRF-PyTorch.

[13] PYNQ Development Team. PYNQ Documentation: DMA[EB/OL]. [2026-10-30]. https://pynq.readthedocs.io/en/latest/pynq_libraries/dma.html.

附录 A 关键接口与寄存器

表 A-1 为当前 PYNQ Overlay 的主要控制地址。软件按匹配的 HWH 查找 IP，地址用于定位与核对。

表 A-1 主要 IP 地址

模块	AXI-Lite 基址
SR-LUT 核心	0x60000000
AXI DMA	0x40400000
显示 VDMA	0x43010000
VTC	0x43C10000

表 A-2 列出 RGBX 计算通路常用寄存器。启动前写入输入尺寸，完成后结合状态、错误码及字数判断本帧执行结果；完整定义以 RTL 包装器和驱动共同维护。

表 A-2 SR-LUT 核心常用寄存器

偏移	名称及作用
0x00	控制：写 bit0 启动，写 bit1 软复位。
0x04 / 0x08	输入宽 / 高；运行期间不改写有效配置。
0x0C	状态：bit0 idle，bit1 busy，bit2 done，bit3 error。
0x10 / 0x14	输入 / 输出字数，用于核对图像尺寸与倍率。
0x18	错误码：尺寸、忙时配置、TLAST 或输出协议异常。
0x1C	输出模式；本报告显示通路使用 RGBX，值为 0。
0x20～0x2C	帧周期、输入背压、输出背压与输入饥饿周期计数。
0x30	输出 CRC32。
0x34	编译时倍率身份，值为 2 或 4，只读。

AXI4-Stream 使用 32 位 RGBX，TKEEP=1111，TLAST 位于整帧末字。接收缓冲长度按输入宽×高×倍率²×4 B 计算，尺寸、传输字数和结束标志需要一致。

附录 B 工程复现与运行方法

B.1 硬件、软件环境与工程准备

准备完整 sr_lut_fpga 工程及其隐藏 .git 目录、PYNQ-Z2、Nexys Video、摄像头、鼠标、供电 USB 集线器、HDMI 线和 MiniDP 至 DP 线。Windows 主机安装 Git、OpenSSH、Vivado 2026.1 与器件支持、Digilent/FTDI 驱动。PYNQ 使用匹配 PYNQ-Z2 的系统镜像，板端已验证环境见表 B-1。

表 B-1 已记录的软件环境

位置	主要环境
Windows host	Python 3.12.14；NumPy 2.2.6；OpenCV 4.12.0；Pillow 11.3.0
Windows reference	Python 3.12.14；NumPy 1.26.4；SciPy 1.12.0；OpenCV 4.10.0
PYNQ 板端	PYNQ 3.1.1；Python 3.10.4；NumPy 1.21.5；OpenCV 4.5.4；Pillow 9.0.1；evdev 1.9.2

主机使用 tools/python.cmd 选择 host 或 reference 环境，首次运行会校验并解压固定依赖到本地缓存。以下命令均从工程根目录执行，示例构建与检查目录须为尚不存在的纯英文路径。Vivado 安装在其他位置时，先设置变量：

```
$env:SR_LUT_VIVADO_BIN='D:\amd\2026.1\Vivado\bin'
```

网络目标默认为 xilinx@192.168.2.99，主机以太网应处于同一网段。新主机配置自己的 SSH 公钥并核实板卡指纹，密钥默认位于用户目录下：

```
.ssh/sr_lut_pynq_z2_ed25519
```

Nexys JTAG 与 UART 的设备身份由工程配置管理，换板时更新真实身份并重新检查。

从新 SD 卡准备环境时，先安装匹配镜像并确认网络、USB 与 PYNQ 驱动，按环境清单准备系统依赖。板端虚拟环境归档、清单与解压目标依次为：

```
runtime/board/environment.json
runtime/board/project_venv_armv7.tar.gz
/home/xilinx/sr_lut_hdmi_envs
```

解压后确认 hdmi_m1_20260909_r11 环境能够导入 pynq、numpy、cv2、PIL 和 evdev。

工程的运行恢复流程依赖原服务基线。按 board/recovery/releases 中保存的发行包安装原服务与候选目录，配置相应软链接，安装快照中的 sr-lut-*.service 和 bootpy.service.d/90-sr-lut-once.conf，重新加载 systemd，启用 portal 服务并保持 candidate 服务停止。

B.2 源码构建与产物检查

先检查第三方 IP，再分别重建 PYNQ x2/x4 与 Nexys。各目标使用独立输出目录，避免构建产物互相覆盖。

```
.\tools\rebuild.ps1 -Target CheckIP
.\tools\rebuild.ps1 -Target PynqX2 -WorkDirectory D:\sr_build_x2
.\tools\rebuild.ps1 -Target PynqX4 -WorkDirectory D:\sr_build_x4
.\tools\rebuild.ps1 -Target NexysSimulation -WorkDirectory D:\sr_build_sim
.\tools\rebuild.ps1 -Target NexysRoute -WorkDirectory D:\sr_build_dp
.\tools\rebuild.ps1 -Target NexysBit `
  -Checkpoint D:\sr_build_dp\route\routed.dcp `
  -WorkDirectory D:\sr_build_bit
```

PYNQ 的 BIT/HWH/XSA 与实现报告分别位于工作目录下的以下位置，x4 对应目录名中的 x2 替换为 x4：

```
source/build/hdmi_x2_1080p30_work/export
source/board/local/hdmi_x2_1080p30
```

Nexys 的路由结果位于 route，位流位于 bit。检查同次构建的 BIT/HWH 配对、IP 参数、地址映射及资源时序报告，确认建立和保持失败端点为零。

LUT 转换脚本、原始模型目录和 ARM 原生像素库的编译配置依次为：

```
data/lut/tools/npy_to_mem.py
data/lut/original_npy
src/ps/hdmi_display/native/build.json
```

转换使用冻结的 x2 q=16 与 x4 q=32 模型；原生库在匹配的 ARM 环境编译。参考与黄金检查可运行：

```
.\tools\python.cmd reference tools/run_tests.py --suite reference `
  --out D:\sr_reference_check
.\tools\python.cmd reference tools/run_tests.py --suite golden `
  --out D:\sr_golden_check
```

```
.\tools\python.cmd host tools/verify_delivery.py
```

参考检查验证参考实现的计算结果，黄金参考检查（golden）比对固定期望输出；交付检查核对文件及清单的一致性。新硬件产物在独立验证后再替换运行配置。VGA 接入的构建、仿真和资格脚本位于独立候选目录，重建参数与 DP 配置分开管理：

```
build/candidates/nexys_vga_26102101
```

B.3 部署、接线与演示运行

当前软件包由 board/release/source.json 指定为 sr_lut_demo_r17_repack.zip，板端目标目录为 /home/xilinx/sr_lut_demo_r17_repack。部署命令复制并核验文件，随后执行本地检查和启动：

```
.\tools\python.cmd host board/host/deploy_business_session.py --run
.\demo.cmd --check
.\demo.cmd
```

按表 2-2 连接 PYNQ HDMI OUT、Nexys HDMI IN 和 DP 显示器，摄像头及鼠标连接到 PYNQ 的供电 USB 集线器。启动后应出现控制终端和显示画面，设备与链路状态正常。使用 VGA 转换器时，入口分别改为 demo_vga.cmd --check 和 demo_vga.cmd，并使用对应显示配置。

表 B-3 给出演示操作顺序。通过图片、视频和摄像头分别观察静态细节、播放控制与现场输入，可以覆盖主要交互路径。

表 B-3 功能运行与观察要点

操作	预期现象
BTN0 图片；SW0 选择倍率	显示图片；x2/x4 切换后尺寸和质量信息更新。
鼠标选择 ROI、Compare 或 Wipe	选区结果更新；分界线移动时显示同位置两种结果。
BTN1 视频	播放、暂停、逐帧、速度和循环控制响应；帧位置更新。
BTN2 摄像头	固定 x4 处理；Freeze 后画面固定，Resume 后继续更新。
保存结果	图片或冻结快照产生 PNG 和 export.json；视频可选择 Export MP4。

B.4 结果核对与正常退出

图片和摄像头保存后，检查 export.json 的来源、倍率、ROI、generation 和 sequence，确认输入与结果尺寸匹配。最近邻应等于输入的整数复制；SR 原始像素可与对应固定 LUT 参考输出比较。质量评分使用原始 PNG，显示缩放和界面叠加不参与评分。

视频导出前确认板端空间充足，保持导出任务所在模式，等待编码与文件写入完成；随后核对 MP4 的帧数、尺寸和时长，并检查关联帧记录。

结束演示时，在原启动终端按一次 Ctrl+C，等待源程序停止、算法 DMA 和显示 DMA 关闭、缓冲释放及原服务和 Nexys 配置恢复，再断开设备。若启动或显示异常，按本地校验、SSH、输入源、HDMI 几何、UART 状态和 DP 训练状态逐项检查；本轮日志位于 board/analysis 或 board/acceptance 对应会话目录。

B.5 Skill 离线运行与迁移入口

把完整 skill 目录复制到独立位置，使用 Python 3.10 或以上版本，在该目录执行以下命令。项目电脑也可用工程的 host Python 代替 python。

```
python -m pip install -r requirements.txt

python run_checks.py --config examples/image_threshold.json `
  --out out/image_threshold

python run_checks.py --config examples/sensor_gain.json `
  --out out/sensor_gain

python run_checks.py --config evidence/project_reports.json `
  --out out/project_reports

python tests/run_tests.py --out out/regression.json

python run_checks.py --config manifest.json --out out/package_integrity
```

成功返回码为 0，失败为 1。例子生成 input.npy、expected.npy、actual.npy、result.json 与 report.md；检查 result.json 中 checks 和 passed，未申请的项目位于 not_requested。回归检查应确认正例通过、损坏输出被检出、同步及生命周期用例通过。

迁移到真实板卡时，按 skill/templates 配置同次构建的 BIT/HWH、DMA 名称和算法 hooks，替换 shape、dtype、字节序、长度与独立参考。先执行最小有效批次和边界批次，再进行重复调用及异常恢复。完成目标板验证后，将输入、期望输出、运行结果和环境记录一起保存，形成该算子的复现入口。